



PES UNIVERSITY
100 feet Ring Road, BSK 3rd Stage, Bengaluru 560085

Department of Computer Science and Engineering
B. Tech. CSE - 5th Semester
Aug – Dec 2026

UE24CS351A
DATABASE MANAGEMENT SYSTEMS (DBMS)

PROJECT REPORT on

JourneyMind: A MultiCommute Decision System

Submitted by: Team Code: 23CS024 - 23CS624 - 23CS366

SRN	Name of the Student
PES1UG23CS024	Adishree Gupta
PES1UG23CS624	Swathi S
PES1UG23CS366	MOHAMMED JAWWAAD SHERIFF

Live Prototype: <https://journeymind.onrender.com/>

Source Code: <https://github.com/1stsimplifycode/JourneyMind-for-DBMS-Project-demo->

JourneyMind: A MultiCommute Decision System*Table of Contents*

Sl. No	Topic	Page No.
1.	Introduction a) Problem Statement and b) Short Description	3
2.	User Requirements Specification	3
3.	E-R Model a) List of Entities [labeling Weak ones] b) List non-simple Attributes c) ER Diagram Snapshot d) ERD from tool like draw.io	6
4.	Relational Model a) List of Relations that are not Entities in ER Model b) Schema Diagram [ensure light background image]	8
5.	SQL DDL Statements	12
6.	SQL DML Statements	18
7.	SQL DCL Statements	29
8.	Results a) Resulting Tables' Screenshots	31
9.	Application Front-end Screenshots	41
10.	Conclusion	48
11.	List of Software Tools Used	49
12.	References, URLs	49

Introduction

Problem Statement

A commuter in Bengaluru usually has several ways of making the same journey: walking, the metro, a bus, an auto, a bike taxi, a cab, or a combination of these. Existing applications show the price of one option at a time, and they show only the advertised price.

That advertised price is often not what the journey actually costs. A booking may find no driver, or a driver may accept and then cancel, and the commuter must try again at a higher fare. The minutes lost waiting are a real cost that no app reports.

The same problem exists at a larger scale for an organisation that pays for staff travel. Across thousands of trips it has no single place to see which campus loses the most time to failed bookings, or which provider is genuinely reliable rather than merely cheap.

JourneyMind addresses both. It compares every practical way of making a journey, judges whether each mode is sensible for the distance before comparing prices, prices the cost of failure, and records every decision so that it can be audited afterwards.

Short Description

JourneyMind is a web application that helps a commuter choose how to travel between two points in a bounded corridor of Bengaluru, and helps an organisation understand the same problem across a large booking history.

The user enters a start and a destination. The system builds the realistic journeys available, including multimodal ones such as bike taxi to metro to walk, and ranks them on cost, time, number of changes and how likely the booking is to succeed. Pressing BOOK NOW simulates a real booking, including the possibility that it fails and must be retried at a higher fare.

The application is built as a Next.js front end talking to a FastAPI back end, which stores data across three databases chosen for the kind of question each is good at answering: MySQL for permanent structured records, Redis for fast temporary data, and Neo4j for the transport network as a graph.

User Requirements Specification

JourneyMind serves two kinds of user: a commuter deciding how to travel, and an analyst studying travel across an organisation. The requirements below are those the implemented system actually satisfies.

User Roles

Role	What this user may do
Commuter (RIDER)	Search journeys, compare options, book a ride, retry a failed booking, view the explanation of a recommendation.
Analyst (ANALYST)	All of the above, plus the organisation dashboard and the audit trail. Requires a valid API key.
Administrator (ADMIN)	Highest privilege level; intended for configuration and key management.

Table 1: User roles and permitted actions

Functional Requirements

ID	Requirement	Description
----	-------------	-------------

JourneyMind: A MultiCommute Decision System

FR-1	Compare travel options	For a given origin and destination, list every practical travel option with its fare, duration and availability.
FR-2	Multimodal journey planning	Build journeys that combine modes, for example bike taxi followed by metro followed by a short walk.
FR-3	Feasibility before ranking	Remove modes that are impractical for the distance before comparing them on price, so a free but unreasonable option cannot win.
FR-4	Expected cost	Show what a journey will really cost once cancellation and re-booking are taken into account, not only the advertised fare.
FR-5	Booking and retry	Simulate a booking, allow up to four attempts, and price each retry according to the surge the market implies.
FR-6	Escalation	When a booking repeatedly fails, warn the user and draft a message they can send.
FR-7	Organisation dashboard	Aggregate the booking history by campus, team, provider and mode, and report the cost of failed bookings.
FR-8	Audit trail	Record every recommendation and enterprise query with the model version and confidence behind it.
FR-9	Data verification	Provide a command that counts every database structure and reports whether it meets the required data volume.

Table 2: Functional requirements

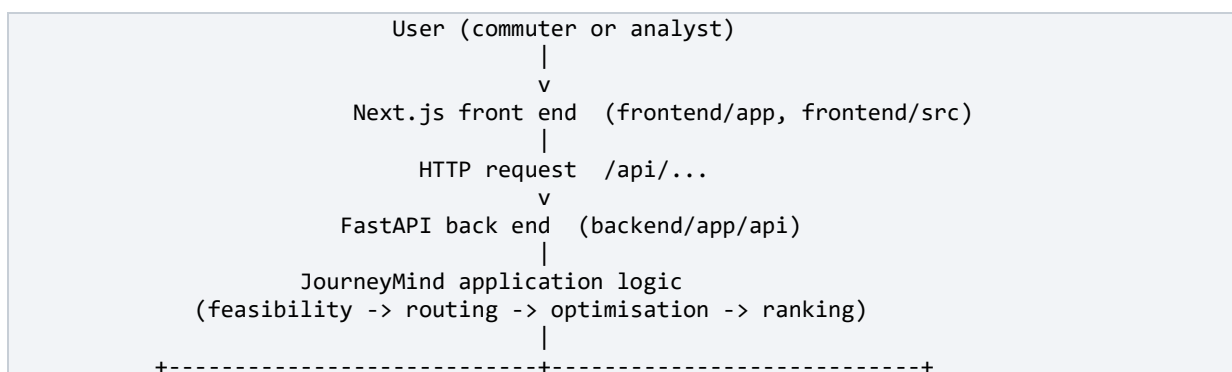
Non-Functional Requirements

ID	Requirement	Description
NFR-1	Security	Database credentials and API keys must never appear in source code. API keys are stored as SHA-256 hashes.
NFR-2	Role-based access	Population-level data is restricted. Three roles exist: RIDER, ANALYST and ADMIN.
NFR-3	Data integrity	Primary keys, foreign keys, UNIQUE and CHECK constraints enforce correctness at the database level.
NFR-4	Graceful degradation	Each database is optional at runtime. If one is unavailable the application continues with a documented fallback.
NFR-5	Reproducibility	A single documented command initialises all three databases and verifies the result.
NFR-6	Testability	Automated tests cover application logic and database integration.

Table 3: Non-functional requirements

System Architecture

The application is built in three layers. The browser never connects to a database directly: every request passes through the FastAPI back end, which holds the database credentials and the application logic.



JourneyMind: A MultiCommute Decision System

 v MySQL permanent structured records and history	 v Redis fast temporary and cached key-value data	 v Neo4j the transport network as a graph
---	---	---

Each database is used for the kind of question it answers well. This choice is justified in Section 4.

JourneyMind: A MultiCommute Decision System

E-R Model

Entity	Type	Attributes	Meaning
Zone	Strong	zone_id (PK), name, kind, lat, lon, latent_congestion, observed_congestion	A pickup or drop area: a metro station, bus stop, named place or junction.
Booking	Strong	booking_id (PK), ts, hour, dow, is_weekend, late_night, rain, provider_id, mode, campus_id, campus, employee_group, cost_centre, zone_id (FK), distance_km, pickup_km, peak_intensity, short_trip_penalty, matched, accepted, cancelled, completed	One historical request for a vehicle and how far it got.
BookingSession	Strong	session_id (PK), created_at, departure, provider_id, display_name, mode, service_class, origin_label, dest_label, advertised_fare, paid_fare, p_match, p_accept, p_cancel, attempt_count, wasted_min, settled, exhausted, demo	One press of BOOK NOW in this application.
BookingAttempt	Weak	attempt_id (PK), session_id (FK), attempt_no, fare, outcome, succeeded, wasted_min, driver_name, eta_min	One try inside a booking session. It cannot exist without its session, and is identified within it by attempt_no.
AuditEvent	Strong	event_id (PK), at, kind, actor, request, decision, model_versions, confidence, data_classes	One recorded decision made by the system.

Table 4: List of entities (weak entity shown in bold)

Non-Simple Attributes

Most attributes are simple. The ones that are not are listed below, with how they are stored.

Attribute	Kind	How it is handled
AuditEvent.request	Composite / structured (JSON)	The question that was asked. Its fields differ between a recommendation, a booking and an enterprise query, so it is stored as a JSON column that MySQL can still query.
AuditEvent.decision	Composite / structured (JSON)	The answer that was given, together with the evidence behind it.
AuditEvent.model_versions	Multivalued (JSON)	The set of model versions that contributed to the decision.
AuditEvent.data_classes	Multivalued (JSON)	The data-sensitivity classes touched by the request.
Zone.lat, Zone.lon	Composite (position)	Together these form the position of the zone; they are stored as two simple columns rather than a composite type.

Table 5: Non-simple attributes

Relationships

Relationship	Between	Cardinality	Description
starts_in	Booking → Zone	Many-to-one (N:1)	Every booking starts in exactly one zone; a zone is the origin of many bookings. Enforced by fk_bookings_zone with ON DELETE RESTRICT.
has_attempt	BookingSession → BookingAttempt	One-to-many (1:N)	One session has between one and four attempts. Enforced by fk_attempts_session with ON DELETE CASCADE, because an attempt has no meaning without its session.

Table 6: Relationships between entities

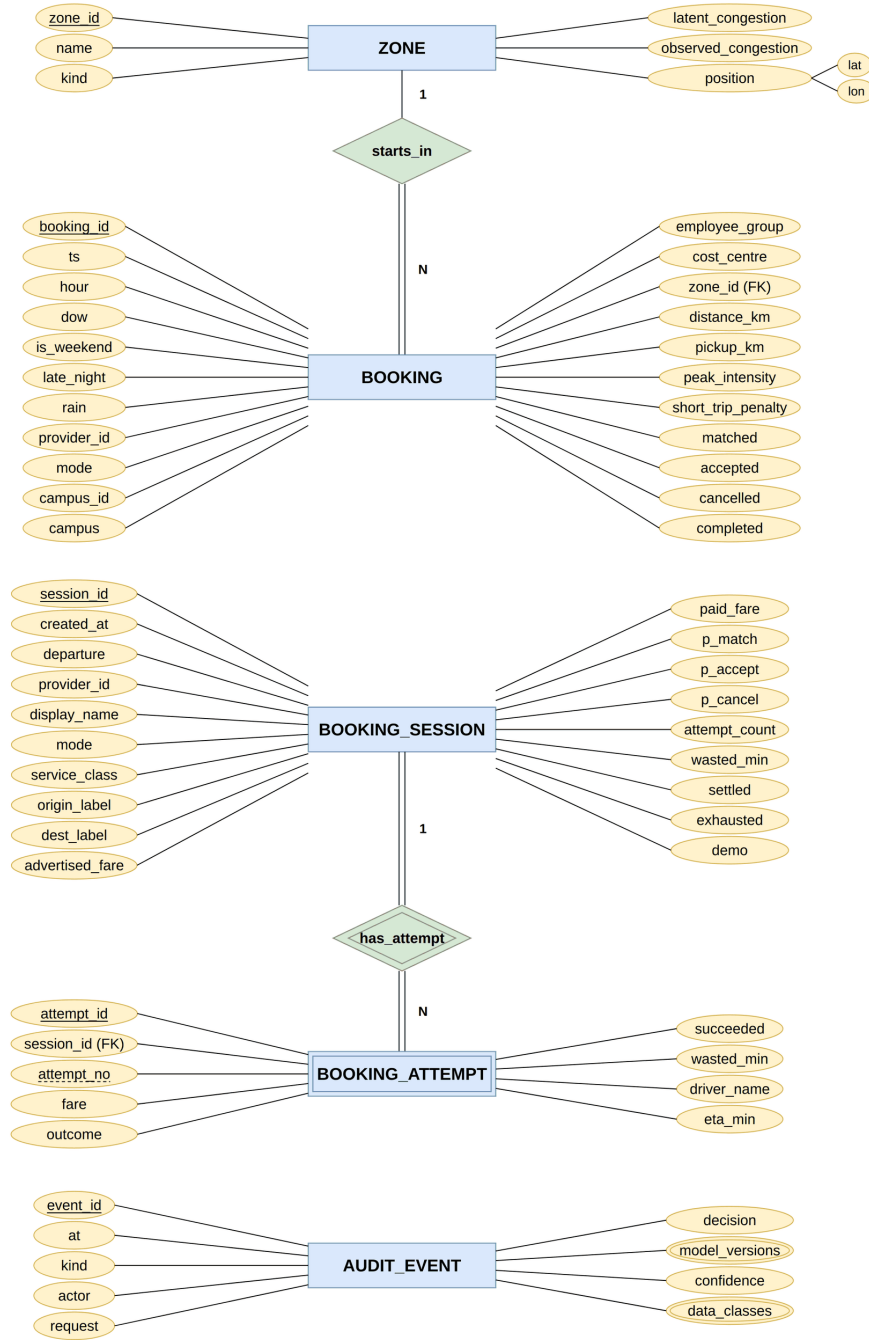
ER Diagram

The diagram below is the E-R model drawn in Chen notation in draw.io. Entities are rectangles, the weak entity a double rectangle, relationships diamonds, the identifying relationship a double diamond and attributes ellipses. Keys are underlined; the full notation is in the figure legend.

JourneyMind: A MultiCommute Decision System

JourneyMind — E-R Diagram (Chen notation)

Database Management Systems project · 23CS024 · 23CS624 · 23CS366



Notation

Entity	Weak entity	Relationship	Identifying relationship	Total participation
Attribute	Multivalued attribute	Composite attribute	Primary key	Partial key

Cardinality is shown Chen-style: BOOKING N — starts_in — 1 ZONE, and BOOKING_SESSION 1 — has_attempt — N BOOKING_ATTEMPT.

Foreign-key attributes (zone_id, session_id) are shown for traceability with Table 4; in strict Chen notation they are implied by the relationship.

AUDIT_EVENT.request and .decision are composite (structured JSON); their inner fields vary by request kind, so each is drawn as a single attribute (Table 5).

Figure 1: E-R diagram in Chen notation, drawn in draw.io

JourneyMind: A MultiCommute Decision System

Relational Model

The E-R model converts to five relations. Four correspond directly to entities. The fifth item listed below, `v_campus_mobility`, is not an entity: it is a view, a saved query over `BOOKINGS` and `ZONES` that stores no rows of its own. No relation here is a many-to-many junction, because both relationships in the model are one-to-many and are represented by a foreign key on the many side.

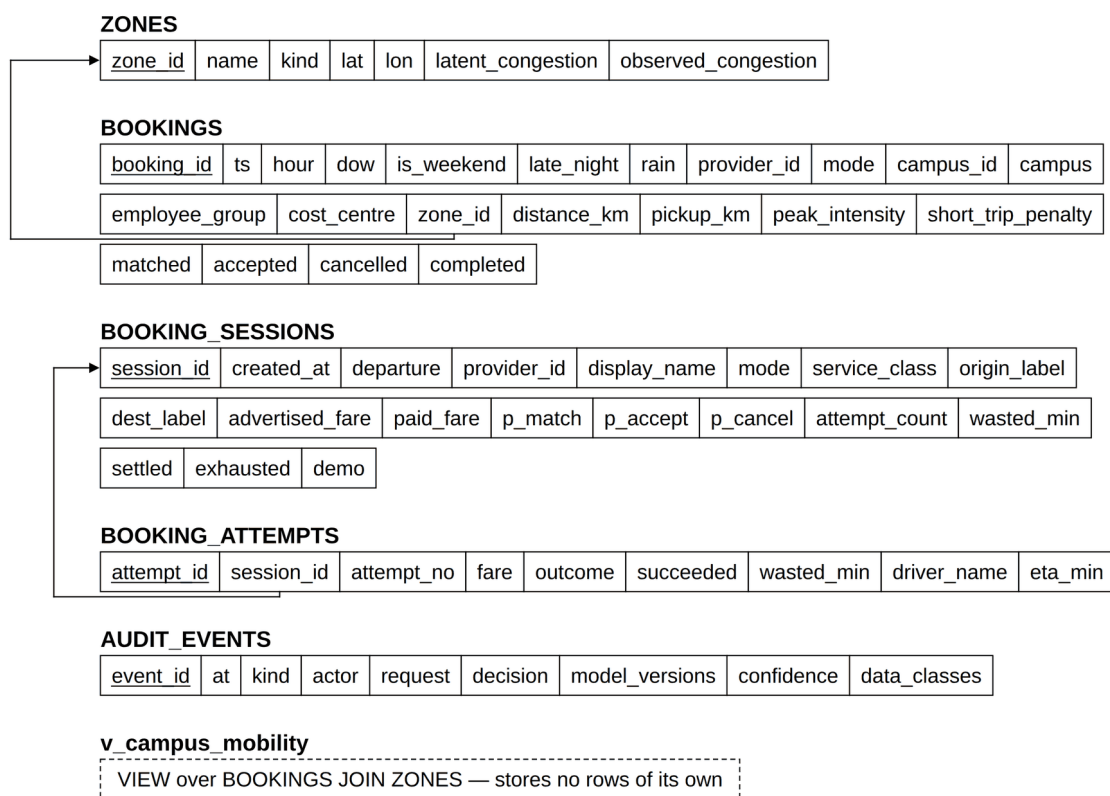
List of Relations

Relation	Corresponds to	Primary key	Foreign keys
ZONES	Zone entity	<u>zone_id</u>	–
BOOKINGS	Booking entity	<u>booking_id</u>	zone_id → ZONES(zone_id)
BOOKING_SESSIONS	BookingSession entity	<u>session_id</u>	–
BOOKING_ATTEMPTS	BookingAttempt (weak entity)	<u>attempt_id</u>	session_id → BOOKING_SESSIONS(session_id)
AUDIT_EVENTS	AuditEvent entity	<u>event_id</u>	–
<code>v_campus_mobility</code>	Not an entity — a view over BOOKINGS and ZONES	–	–

Table 7: List of relations, with the view identified as not an entity in the E-R model

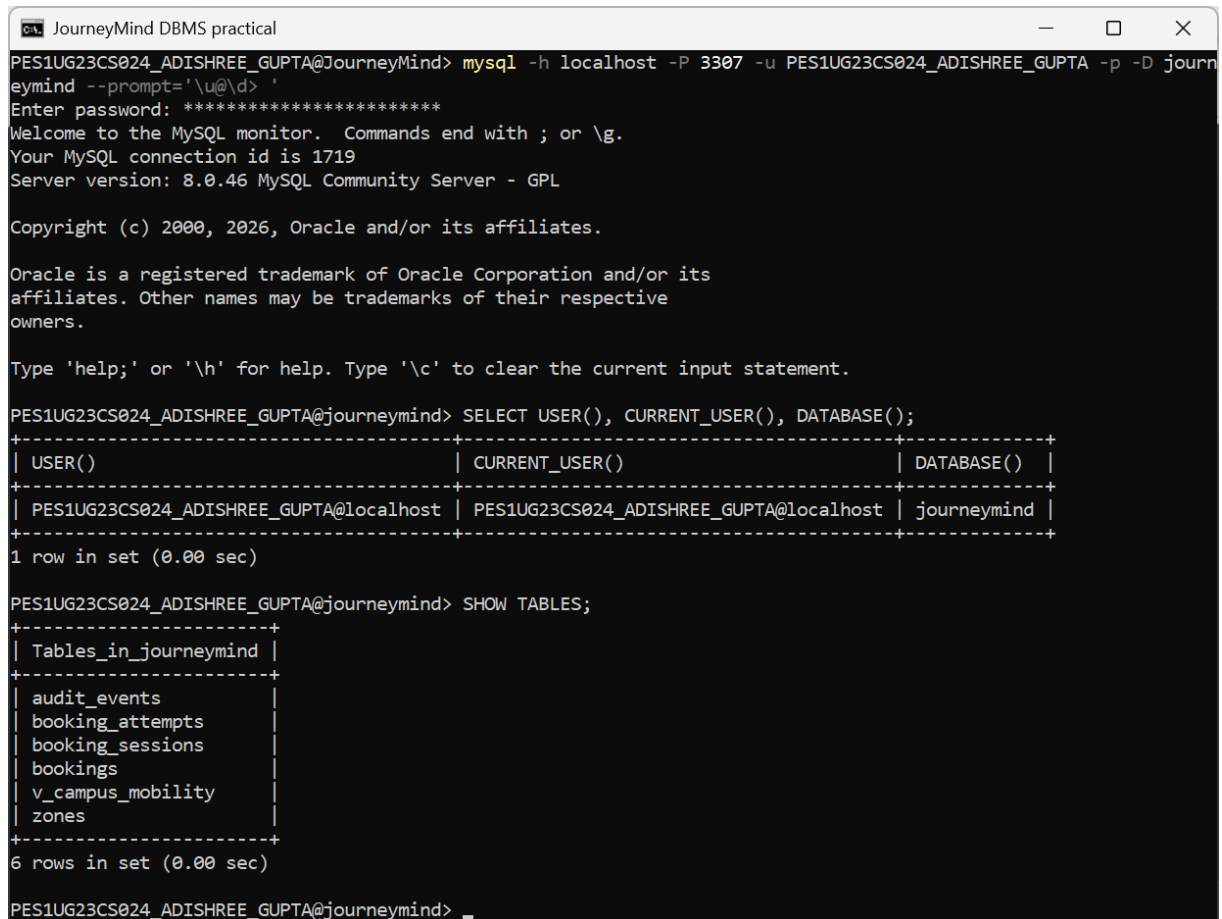
Schema Diagram

Primary keys are underlined and an arrow runs from each foreign key to the primary key it references.



Referential actions: zone_id → ZONES(zone_id) ON DELETE RESTRICT · session_id → BOOKING_SESSIONS(session_id) ON DELETE CASCADE · UNIQUE (session_id, attempt_no) on BOOKING_ATTEMPTS

JourneyMind: A MultiCommute Decision System



```

C:\> JourneyMind DBMS practical
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind> mysql -h localhost -P 3307 -u PES1UG23CS024_ADISHREE_GUPTA -p -D journeymind --prompt='\u@d> '
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 1719
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

PES1UG23CS024_ADISHREE_GUPTA@journeymind> SELECT USER(), CURRENT_USER(), DATABASE();
+-----+-----+-----+
| USER()                | CURRENT_USER()                | DATABASE() |
+-----+-----+-----+
| PES1UG23CS024_ADISHREE_GUPTA@localhost | PES1UG23CS024_ADISHREE_GUPTA@localhost | journeymind |
+-----+-----+-----+
1 row in set (0.00 sec)

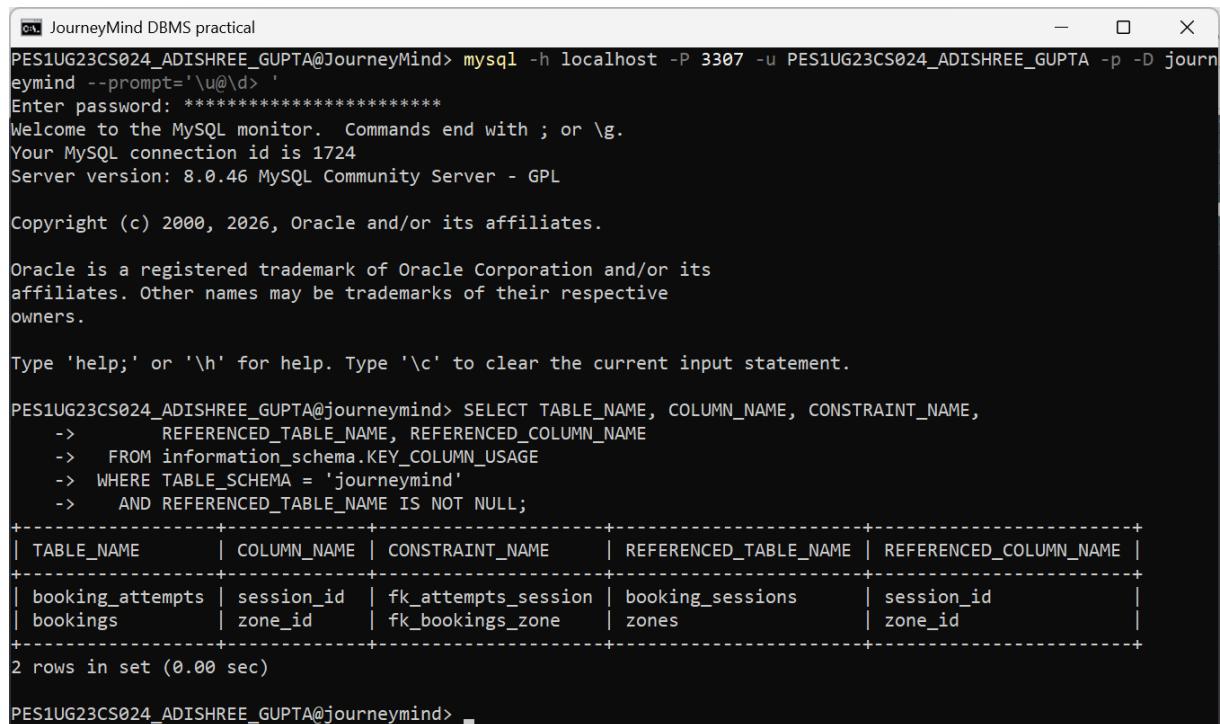
PES1UG23CS024_ADISHREE_GUPTA@journeymind> SHOW TABLES;
+-----+
| Tables_in_journeymind |
+-----+
| audit_events          |
| booking_attempts      |
| booking_sessions      |
| bookings              |
| v_campus_mobility     |
| zones                 |
+-----+
6 rows in set (0.00 sec)

PES1UG23CS024_ADISHREE_GUPTA@journeymind> _

```

Figure 2: The five base tables and the reporting view, as created in MySQL

JourneyMind: A MultiCommute Decision System



```

JourneyMind DBMS practical
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind> mysql -h localhost -P 3307 -u PES1UG23CS024_ADISHREE_GUPTA -p -D journeymind --prompt='\u@\d> '
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 1724
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

PES1UG23CS024_ADISHREE_GUPTA@journeymind> SELECT TABLE_NAME, COLUMN_NAME, CONSTRAINT_NAME,
->         REFERENCED_TABLE_NAME, REFERENCED_COLUMN_NAME
->   FROM information_schema.KEY_COLUMN_USAGE
->  WHERE TABLE_SCHEMA = 'journeymind'
->        AND REFERENCED_TABLE_NAME IS NOT NULL;
+-----+-----+-----+-----+-----+
| TABLE_NAME | COLUMN_NAME | CONSTRAINT_NAME | REFERENCED_TABLE_NAME | REFERENCED_COLUMN_NAME |
+-----+-----+-----+-----+-----+
| booking_attempts | session_id | fk_attempts_session | booking_sessions | session_id |
| bookings | zone_id | fk_bookings_zone | zones | zone_id |
+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)

PES1UG23CS024_ADISHREE_GUPTA@journeymind>

```

Figure 3: The two foreign keys, read from information_schema

Database Selection and Justification

The project uses one SQL database and two NoSQL databases. Each was chosen because of the kind of question the data is asked, not to demonstrate a technology.

Question the application asks	Database	Why
How much did this campus spend last month, excluding cancelled trips?	MySQL	A filtered aggregate over 60,000 related rows. This is what tables, joins, indexes and SQL exist for.
What is the value stored under this exact key, right now?	Redis	A single lookup by key, on data that is temporary. No search and no relationships are involved.
Which stops connect to which, and how many routes meet here?	Neo4j	A question about connections. Following relationships is what a graph database does naturally.

Table 8: Database selection by access pattern

MySQL: structured, permanent, related records

- Stores zones, the 60,000-row booking history, the bookings this instance makes, the attempts inside them, and the audit trail.
- The data is strongly related: a booking names a zone, an attempt belongs to a session. Foreign keys enforce this.
- The enterprise dashboard filters and totals these rows by campus, team, provider and date, which is exactly a relational workload.
- Redis would be the wrong choice because there is no single key such a question could be looked up by; the filters are chosen at query time.

Redis: fast, temporary, key-based data

- Stores cached place lookups, live booking sessions, cached dashboard answers and a bounded list of recent decisions.
- Each item is read by an exact key and most of it is short-lived, so it is given a time limit (TTL) after which Redis removes it.
- Holding live booking sessions in MySQL would mean a table whose rows are all deleted within the hour, plus a job to delete them.

Neo4j: the transport network as connections

- Stores the corridor as 223 Stop nodes joined by 1,442 relationships of three kinds: road links, transit links and walking transfers.
- The application asks it which stops are served by two or more routes, and the ordered sequence of stops along each route.
- The same question in SQL means grouping an edge table and counting distinct routes; in a graph the connections are the data.

Redis compared with Neo4j

These two are both NoSQL databases but answer different kinds of question, and this distinction matters.

	Redis	Neo4j
The question it answers	I know the exact key. Give me its value, quickly.	This thing is connected to what?
JourneyMind example	Look up jm:geo:cubbon park and get its coordinates back in one operation.	Start at the stop Majestic and follow its TRANSIT_LINK relationships to find a path across the city.
Does it follow links?	No. There is no concept of one record referring to another.	Yes. Following relationships cheaply is its entire purpose.
Data lifetime	Often temporary, controlled by a TTL.	Permanent; rebuilt only when the network data is re-seeded.

Table 9: Redis compared with Neo4j

Note on overlap between MySQL and Neo4j

The 105 zones in MySQL and the Stop nodes in Neo4j describe some of the same physical places. This is deliberate rather than accidental duplication: both are loaded from the same bundled source files, neither is edited while the application runs, and they are modelled for two different questions. A zone is a lookup dimension that bookings point at; a stop is a vertex in a network that can be traversed. Data that the application writes is never duplicated: a booking record exists only in MySQL, a live session only in Redis, and nothing is ever written into Neo4j by a booking.

SQL DDL Statements

DDL (Data Definition Language) statements define the structure of the database. The statements below are the project's own, taken from database/mysql/schema.sql, which is executed by database/seed_mysql.py.

CREATE TABLE with Primary Key and Constraints

```
CREATE TABLE IF NOT EXISTS zones (
  zone_id          VARCHAR(64)  NOT NULL,
  name             VARCHAR(128) NOT NULL,
  kind             ENUM('metro_station','bus_stop','place','junction') NOT NULL,
  lat              DECIMAL(9, 6) NOT NULL,
  lon              DECIMAL(9, 6) NOT NULL,
  latent_congestion DECIMAL(6, 5) NOT NULL DEFAULT 0,
  observed_congestion DECIMAL(6, 5) NOT NULL DEFAULT 0,
  PRIMARY KEY (zone_id),
  KEY idx_zones_kind (kind),
  CONSTRAINT chk_zones_lat CHECK (lat BETWEEN -90 AND 90),
  CONSTRAINT chk_zones_lon CHECK (lon BETWEEN -180 AND 180)
) ENGINE = InnoDB;
```

zone_id is the primary key. The CHECK constraints refuse any latitude or longitude outside its physical range, so impossible coordinates cannot be stored.

CREATE TABLE with Foreign Key and a Composite UNIQUE Key

```
CREATE TABLE IF NOT EXISTS booking_attempts (
  attempt_id      BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  session_id      VARCHAR(32)     NOT NULL,
  attempt_no      TINYINT         NOT NULL,
  fare            DECIMAL(10, 2)  NOT NULL,
  outcome         VARCHAR(32)     NOT NULL,
  succeeded       BOOLEAN         NOT NULL DEFAULT FALSE,
  wasted_min      DECIMAL(7, 2)   NOT NULL DEFAULT 0,
  driver_name     VARCHAR(64)     NULL,
  eta_min         DECIMAL(6, 2)   NULL,
  PRIMARY KEY (attempt_id),
  UNIQUE KEY uq_attempt_per_session (session_id, attempt_no),
  CONSTRAINT fk_attempts_session FOREIGN KEY (session_id)
    REFERENCES booking_sessions (session_id) ON DELETE CASCADE,
  CONSTRAINT chk_attempts_no CHECK (attempt_no >= 1),
  CONSTRAINT chk_attempts_fare CHECK (fare >= 0)
) ENGINE = InnoDB;
```

- **PRIMARY KEY (attempt_id)** uniquely identifies each attempt.
- **FOREIGN KEY (session_id)** ties every attempt to a real booking session. ON DELETE CASCADE is correct here because an attempt has no meaning once its session is gone.
- **UNIQUE (session_id, attempt_no)** prevents two rows both claiming to be attempt 2 of the same booking.

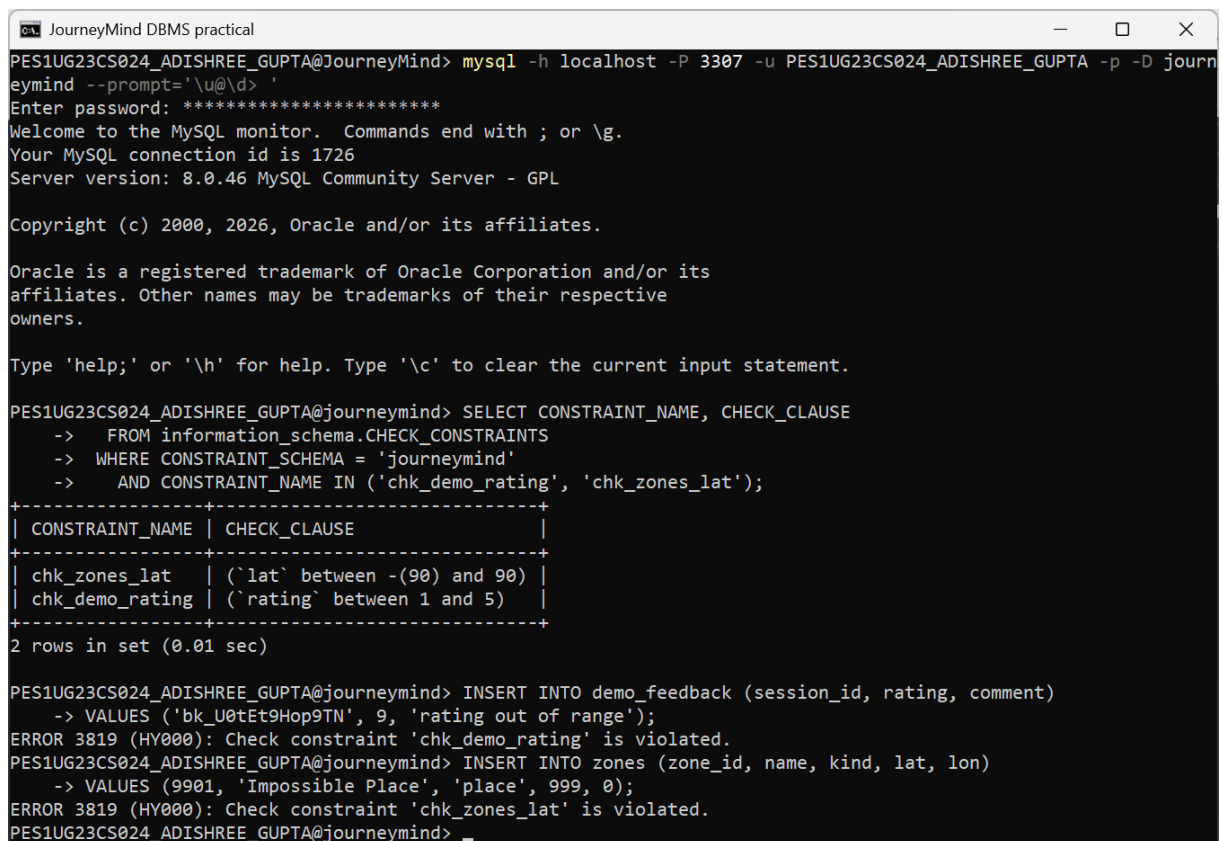
JourneyMind: A MultiCommute Decision System

Referential Integrity and a Business Rule as a Constraint

```
-- Referential integrity and a rule the lifecycle must obey
CONSTRAINT fk_bookings_zone FOREIGN KEY (zone_id)
    REFERENCES zones (zone_id) ON DELETE RESTRICT ON UPDATE CASCADE,

CONSTRAINT chk_bookings_lifecycle CHECK (
    (accepted = FALSE OR matched = TRUE)
    AND (completed = FALSE OR accepted = TRUE)
    AND NOT (completed = TRUE AND cancelled = TRUE));
```

ON DELETE RESTRICT is used here rather than CASCADE: silently deleting sixty thousand historical facts because a zone row was removed would not be a repair. The lifecycle CHECK encodes a real rule of the domain, that a trip cannot be completed unless it was accepted, and cannot be accepted unless a driver was matched.



```
JourneyMind DBMS practical
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind> mysql -h localhost -P 3307 -u PES1UG23CS024_ADISHREE_GUPTA -p -D journeymind --prompt='\u@>'
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 1726
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

PES1UG23CS024_ADISHREE_GUPTA@journeymind> SELECT CONSTRAINT_NAME, CHECK_CLAUSE
  -> FROM information_schema.CHECK_CONSTRAINTS
  -> WHERE CONSTRAINT_SCHEMA = 'journeymind'
  -> AND CONSTRAINT_NAME IN ('chk_demo_rating', 'chk_zones_lat');
+-----+-----+
| CONSTRAINT_NAME | CHECK_CLAUSE |
+-----+-----+
| chk_zones_lat   | ('lat' between -90 and 90) |
| chk_demo_rating | ('rating' between 1 and 5) |
+-----+-----+
2 rows in set (0.01 sec)

PES1UG23CS024_ADISHREE_GUPTA@journeymind> INSERT INTO demo_feedback (session_id, rating, comment)
  -> VALUES ('bk_U0tEt9Hop9TN', 9, 'rating out of range');
ERROR 3819 (HY000): Check constraint 'chk_demo_rating' is violated.
PES1UG23CS024_ADISHREE_GUPTA@journeymind> INSERT INTO zones (zone_id, name, kind, lat, lon)
  -> VALUES (9901, 'Impossible Place', 'place', 999, 0);
ERROR 3819 (HY000): Check constraint 'chk_zones_lat' is violated.
PES1UG23CS024_ADISHREE_GUPTA@journeymind>
```

Figure 4: The CHECK constraints, and MySQL refusing two rows that break them

Indexes

KEY idx_bookings_ts	(ts),
KEY idx_bookings_zone	(zone_id),
KEY idx_bookings_hour	(hour),
KEY idx_bookings_campus_provider	(campus_id, provider_id),
KEY idx_bookings_group	(employee_group),
KEY idx_audit_at	(at),
KEY idx_audit_kind_at	(kind, at)

JourneyMind: A MultiCommute Decision System

Each index matches a filter the dashboard actually applies. `idx_bookings_campus_provider` is a composite index serving the common drill-down of one campus and one provider together.

```

JourneyMind DBMS practical
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind> mysql -h localhost -P 3307 -u PES1UG23CS024_ADISHREE_GUPTA -p -D journeymind --prompt='\u@\d> '
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 1727
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

PES1UG23CS024_ADISHREE_GUPTA@journeymind> SELECT TABLE_NAME, INDEX_NAME, SEQ_IN_INDEX, COLUMN_NAME
-> FROM information_schema.STATISTICS
-> WHERE TABLE_SCHEMA = 'journeymind'
-> AND TABLE_NAME = 'bookings'
-> ORDER BY INDEX_NAME, SEQ_IN_INDEX;
+-----+-----+-----+-----+
| TABLE_NAME | INDEX_NAME | SEQ_IN_INDEX | COLUMN_NAME |
+-----+-----+-----+-----+
| bookings | idx_bookings_campus_provider | 1 | campus_id |
| bookings | idx_bookings_campus_provider | 2 | provider_id |
| bookings | idx_bookings_group | 1 | employee_group |
| bookings | idx_bookings_hour | 1 | hour |
| bookings | idx_bookings_ts | 1 | ts |
| bookings | idx_bookings_zone | 1 | zone_id |
| bookings | PRIMARY | 1 | booking_id |
+-----+-----+-----+-----+
7 rows in set (0.00 sec)

PES1UG23CS024_ADISHREE_GUPTA@journeymind>

```

Figure 5: Indexes created on the JourneyMind tables

CREATE VIEW

```

CREATE OR REPLACE VIEW v_campus_mobility AS
SELECT b.campus_id, b.campus, z.kind AS zone_kind,
       COUNT(*) AS bookings,
       SUM(b.completed) AS completed_trips,
       SUM(b.cancelled) AS cancelled_trips,
       ROUND(AVG(b.distance_km), 2) AS avg_distance_km,
       ROUND(100.0 * SUM(b.completed) / COUNT(*), 1) AS completion_pct,
       ROUND(100.0 * SUM(b.cancelled) / COUNT(*), 1) AS cancellation_pct
FROM bookings b
JOIN zones z ON z.zone_id = b.zone_id
GROUP BY b.campus_id, b.campus, z.kind;

```

A view is a stored query, not stored data. Reading it runs the join and grouping underneath against the live tables, so it can never hold a stale copy.

ALTER and DROP

ALTER and DROP are demonstrated against a temporary table created for the purpose, so that the project's real data is never placed at risk.

```
-- Demonstrated on a throwaway table so project data is never at risk
CREATE TABLE demo_feedback (
    feedback_id INT AUTO_INCREMENT PRIMARY KEY,
    session_id VARCHAR(32) NOT NULL,
    rating TINYINT NOT NULL,
    comment VARCHAR(200),
    CONSTRAINT chk_demo_rating CHECK (rating BETWEEN 1 AND 5)
) ENGINE = InnoDB;

ALTER TABLE demo_feedback ADD COLUMN created_at DATETIME NULL;

CREATE INDEX idx_demo_rating ON demo_feedback (rating);

DROP TABLE IF EXISTS demo_feedback;
```

JourneyMind: A MultiCommute Decision System

```

JourneyMind DBMS practical
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind> mysql -h localhost -P 3307 -u PES1UG23CS024_ADISHREE_GUPTA -p -D journeymind --prompt='\u@> '
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 1743
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

PES1UG23CS024_ADISHREE_GUPTA@journeymind> DROP TABLE IF EXISTS demo_feedback;
Query OK, 0 rows affected (0.02 sec)

PES1UG23CS024_ADISHREE_GUPTA@journeymind> CREATE TABLE demo_feedback (
  -> feedback_id INT AUTO_INCREMENT PRIMARY KEY,
  -> session_id VARCHAR(32) NOT NULL,
  -> rating TINYINT NOT NULL,
  -> comment VARCHAR(200),
  -> CONSTRAINT chk_demo_rating CHECK (rating BETWEEN 1 AND 5)
  -> ) ENGINE = InnoDB;
Query OK, 0 rows affected (0.05 sec)

PES1UG23CS024_ADISHREE_GUPTA@journeymind> ALTER TABLE demo_feedback ADD COLUMN created_at DATETIME NULL;
Query OK, 0 rows affected (0.07 sec)
Records: 0 Duplicates: 0 Warnings: 0

PES1UG23CS024_ADISHREE_GUPTA@journeymind> CREATE INDEX idx_demo_rating ON demo_feedback (rating);
Query OK, 0 rows affected (0.03 sec)
Records: 0 Duplicates: 0 Warnings: 0

PES1UG23CS024_ADISHREE_GUPTA@journeymind> DESCRIBE demo_feedback;
+-----+-----+-----+-----+-----+-----+
| Field      | Type      | Null | Key | Default | Extra      |
+-----+-----+-----+-----+-----+-----+
| feedback_id | int       | NO   | PRI | NULL    | auto_increment |
| session_id  | varchar(32) | NO   |     | NULL    |              |
| rating      | tinyint   | NO   | MUL | NULL    |              |
| comment     | varchar(200) | YES  |     | NULL    |              |
| created_at  | datetime  | YES  |     | NULL    |              |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)

PES1UG23CS024_ADISHREE_GUPTA@journeymind>

```

Figure 6: CREATE, ALTER, CREATE INDEX and DROP executing against MySQL

Schema Definition in Neo4j

A graph database does not require a fixed schema, but Neo4j supports a uniqueness constraint and indexes, and JourneyMind uses both. The constraint is the graph equivalent of a key constraint.

```

// Neo4j schema: a uniqueness constraint and two indexes
CREATE CONSTRAINT stop_id_unique IF NOT EXISTS
FOR (s:Stop) REQUIRE s.stop_id IS UNIQUE;

CREATE INDEX stop_kind IF NOT EXISTS
FOR (s:Stop) ON (s.kind);

```

JourneyMind: A MultiCommute Decision System

```
CREATE INDEX transit_route IF NOT EXISTS  
FOR ()-[r:TRANSIT_LINK]-() ON (r.route_id);
```

SQL DML Statements

DML (Data Manipulation Language) statements work with the data inside the tables. All four operations are used by the running application.

INSERT

```
-- Loading the bundled zone data. ON DUPLICATE KEY UPDATE rather than
-- REPLACE, because REPLACE deletes the row first and the foreign key from
-- bookings correctly refuses that.
INSERT INTO zones
  (zone_id, name, kind, lat, lon, latent_congestion, observed_congestion)
VALUES (%s, %s, %s, %s, %s, %s, %s)
ON DUPLICATE KEY UPDATE
  name = VALUES(name), kind = VALUES(kind),
  lat = VALUES(lat), lon = VALUES(lon),
  latent_congestion = VALUES(latent_congestion),
  observed_congestion = VALUES(observed_congestion);

-- One attempt row per press of BOOK NOW or TRY AGAIN
INSERT INTO booking_attempts
  (session_id, attempt_no, fare, outcome, succeeded,
   wasted_min, driver_name, eta_min)
VALUES (%s, %s, %s, %s, %s, %s, %s, %s);
```

UPDATE and DELETE

```
-- UPDATE: correct a rating
UPDATE demo_feedback
  SET rating = 3, comment = 'Retried, but got there'
  WHERE rating = 2;

-- DELETE: a session is re-saved, so its previous attempt rows are cleared
DELETE FROM booking_attempts WHERE session_id = %s;
```

JourneyMind: A MultiCommute Decision System

```

PES1UG23CS024_ADISHREE_GUPTA@JourneyMind> mysql -h localhost -P 3307 -u PES1UG23CS024_ADISHREE_GUPTA -p -D journeymind --prompt='\u@> '
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 1763
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

PES1UG23CS024_ADISHREE_GUPTA@journeymind> INSERT INTO demo_feedback (session_id, rating, comment)
-> VALUES ('bk_U0tEt9Hop9TN', 5, 'Bike taxi arrived quickly'),
-> ('bk_e1WVYt7azsgB', 2, 'Had to retry three times');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

PES1UG23CS024_ADISHREE_GUPTA@journeymind> SELECT feedback_id, rating, comment FROM demo_feedback;
+-----+-----+-----+
| feedback_id | rating | comment                |
+-----+-----+-----+
| 1           | 5      | Bike taxi arrived quickly |
| 2           | 2      | Had to retry three times  |
+-----+-----+-----+
2 rows in set (0.00 sec)

PES1UG23CS024_ADISHREE_GUPTA@journeymind> UPDATE demo_feedback SET rating = 3,
-> comment = 'Retried, but got there'
-> WHERE rating = 2;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

PES1UG23CS024_ADISHREE_GUPTA@journeymind> SELECT feedback_id, rating, comment FROM demo_feedback;
+-----+-----+-----+
| feedback_id | rating | comment                |
+-----+-----+-----+
| 1           | 5      | Bike taxi arrived quickly |
| 2           | 3      | Retried, but got there    |
+-----+-----+-----+
2 rows in set (0.00 sec)

PES1UG23CS024_ADISHREE_GUPTA@journeymind> DELETE FROM demo_feedback WHERE rating = 3;
Query OK, 1 row affected (0.00 sec)

PES1UG23CS024_ADISHREE_GUPTA@journeymind>

```

Figure 7: INSERT, SELECT, UPDATE and DELETE executing in the MySQL client

Every reply above comes from MySQL itself: the row counts, the "Query OK" lines and the result grids. Figure 4 shows the other half of the story, where the server refuses a rating of 9 because it breaks CHECK (rating BETWEEN 1 AND 5). Validation is enforced by the database, not only by the application.

SELECT with WHERE and ORDER BY

Question: which completed trips were the longest?

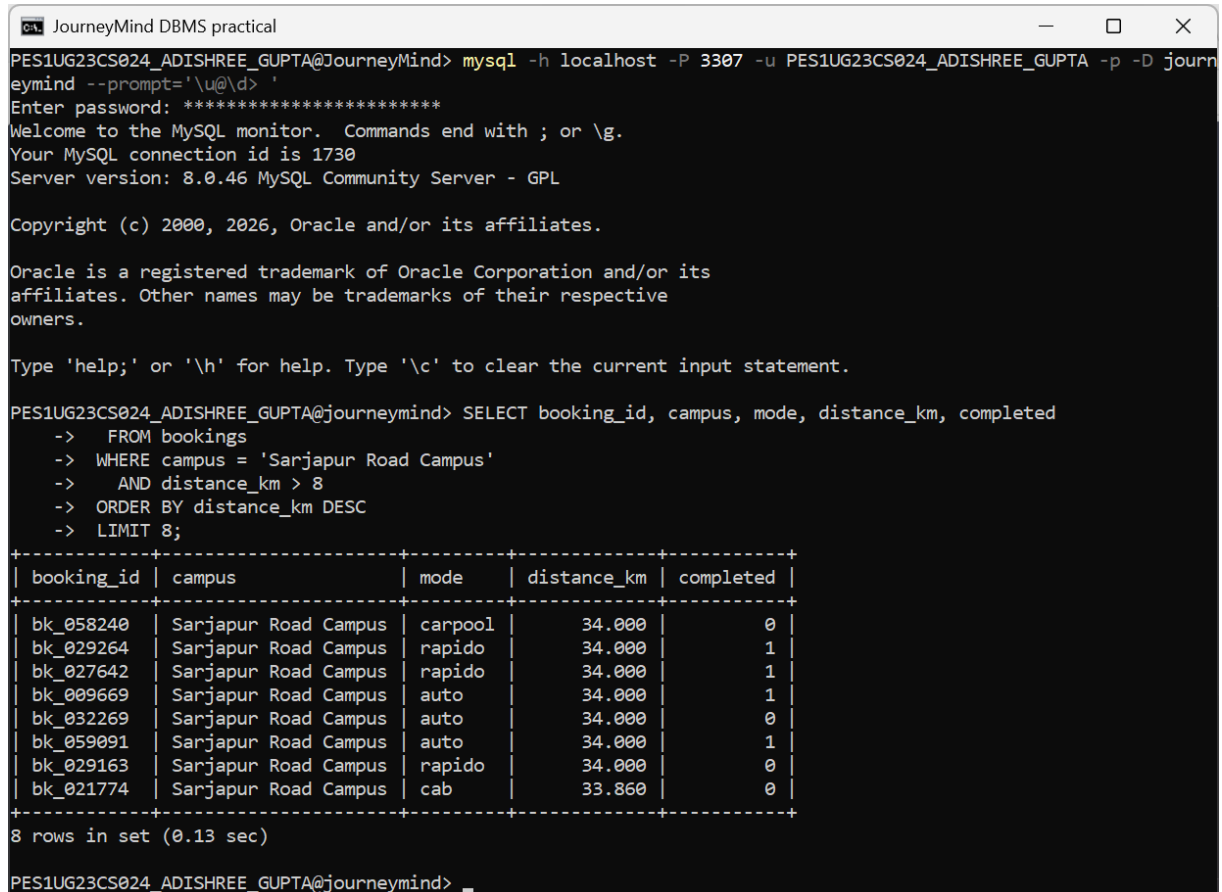
```

SELECT booking_id, campus, mode,
       ROUND(distance_km, 2) AS distance_km, completed
FROM bookings
WHERE completed = TRUE

```

JourneyMind: A MultiCommute Decision System

```
ORDER BY distance_km DESC
LIMIT 8;
```



```
JourneyMind DBMS practical
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind> mysql -h localhost -P 3307 -u PES1UG23CS024_ADISHREE_GUPTA -p -D journeymind --prompt='\u@\d> '
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 1730
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

PES1UG23CS024_ADISHREE_GUPTA@journeymind> SELECT booking_id, campus, mode, distance_km, completed
-> FROM bookings
-> WHERE campus = 'Sarjapur Road Campus'
-> AND distance_km > 8
-> ORDER BY distance_km DESC
-> LIMIT 8;
+-----+-----+-----+-----+-----+
| booking_id | campus | mode | distance_km | completed |
+-----+-----+-----+-----+-----+
| bk_058240 | Sarjapur Road Campus | carpool | 34.000 | 0 |
| bk_029264 | Sarjapur Road Campus | rapido | 34.000 | 1 |
| bk_027642 | Sarjapur Road Campus | rapido | 34.000 | 1 |
| bk_009669 | Sarjapur Road Campus | auto | 34.000 | 1 |
| bk_032269 | Sarjapur Road Campus | auto | 34.000 | 0 |
| bk_059091 | Sarjapur Road Campus | auto | 34.000 | 1 |
| bk_029163 | Sarjapur Road Campus | rapido | 34.000 | 0 |
| bk_021774 | Sarjapur Road Campus | cab | 33.860 | 0 |
+-----+-----+-----+-----+-----+
8 rows in set (0.13 sec)

PES1UG23CS024_ADISHREE_GUPTA@journeymind>
```

Figure 8: SELECT with WHERE and ORDER BY

JOIN

Question: for each booking, what kind of place did it start from? The booking table stores only a zone code; the join follows it into the zones table.

```
SELECT b.booking_id, b.campus,
       z.name AS zone_name, z.kind AS zone_kind
FROM bookings b
JOIN zones z ON z.zone_id = b.zone_id
LIMIT 8;
```

JourneyMind: A MultiCommute Decision System

```

C:\> JourneyMind DBMS practical
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind> mysql -h localhost -P 3307 -u PES1UG23CS024_ADISHREE_GUPTA -p -D journeymind --prompt='\u@\d> '
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 1736
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

PES1UG23CS024_ADISHREE_GUPTA@journeymind> SELECT b.booking_id, b.campus, z.name AS zone_name,
->      z.kind, b.distance_km
->      FROM bookings b
->      JOIN zones z ON z.zone_id = b.zone_id
->      WHERE b.completed = TRUE
->      ORDER BY b.booking_id
->      LIMIT 8;
+-----+-----+-----+-----+-----+
| booking_id | campus | zone_name | kind | distance_km |
+-----+-----+-----+-----+-----+
| bk_000000 | Sarjapur Road Campus | Wipro Campus, Doddakannelli (Sarjapur Road) | place | 3.283 |
| bk_000003 | MG Road Office | M.G. Road | place | 3.589 |
| bk_000004 | Sarjapur Road Campus | Kanakapura Road stop 7 | bus_stop | 1.753 |
| bk_000006 | Electronic City Hub | Bommanahalli | metro_station | 5.506 |
| bk_000007 | Old Airport Road Site | Old Airport Road Gate | place | 0.600 |
| bk_000008 | Sarjapur Road Campus | Wipro Campus, Doddakannelli (Sarjapur Road) | place | 1.235 |
| bk_000009 | Electronic City Hub | Bommanahalli | metro_station | 1.726 |
| bk_000010 | Koramangala Annexe | Outer Ring Road stop 8 | bus_stop | 1.745 |
+-----+-----+-----+-----+-----+
8 rows in set (0.00 sec)

PES1UG23CS024_ADISHREE_GUPTA@journeymind>

```

Figure 9: A JOIN between bookings and zones

GROUP BY, HAVING and Aggregate Functions

Question: for each busy campus, how many trips were made and how far was the average one? This uses five aggregate functions, groups the rows by campus, and then filters the groups with HAVING.

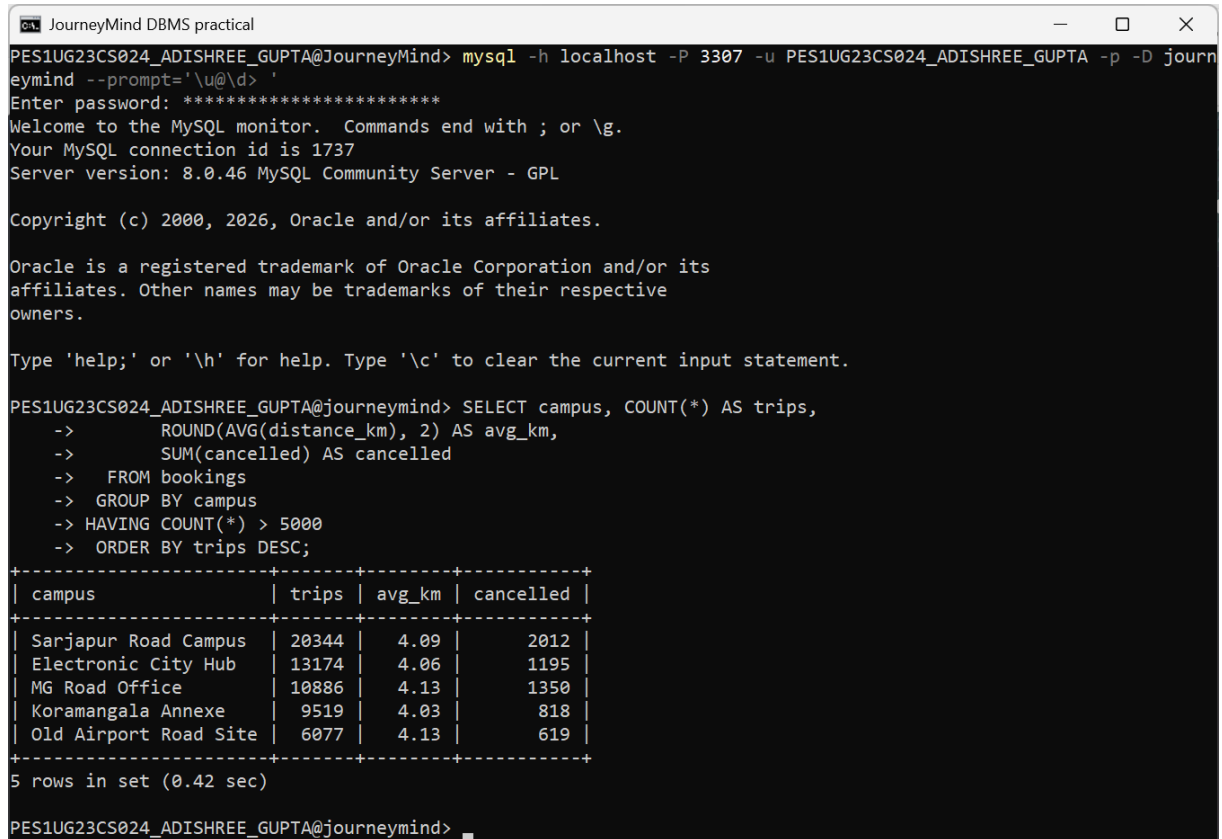
```

SELECT campus,
       COUNT(*) AS bookings,
       ROUND(AVG(distance_km),2) AS avg_km,
       ROUND(MIN(distance_km),2) AS min_km,
       ROUND(MAX(distance_km),2) AS max_km,
       SUM(completed) AS completed
FROM bookings

```

JourneyMind: A MultiCommute Decision System

```
GROUP BY campus
HAVING COUNT(*) > 5000
ORDER BY bookings DESC;
```



```
JourneyMind DBMS practical
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind> mysql -h localhost -P 3307 -u PES1UG23CS024_ADISHREE_GUPTA -p -D journeymind --prompt='\u@\d> '
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 1737
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

PES1UG23CS024_ADISHREE_GUPTA@journeymind> SELECT campus, COUNT(*) AS trips,
->      ROUND(AVG(distance_km), 2) AS avg_km,
->      SUM(cancelled) AS cancelled
->      FROM bookings
->      GROUP BY campus
->      HAVING COUNT(*) > 5000
->      ORDER BY trips DESC;
+-----+-----+-----+-----+
| campus                | trips | avg_km | cancelled |
+-----+-----+-----+-----+
| Sarjapur Road Campus  | 20344 | 4.09   | 2012      |
| Electronic City Hub    | 13174 | 4.06   | 1195      |
| MG Road Office         | 10886 | 4.13   | 1350      |
| Koramangala Annexe     | 9519  | 4.03   | 818       |
| Old Airport Road Site  | 6077  | 4.13   | 619       |
+-----+-----+-----+-----+
5 rows in set (0.42 sec)

PES1UG23CS024_ADISHREE_GUPTA@journeymind> _
```

Figure 10: GROUP BY with HAVING and the aggregate functions COUNT, AVG, MIN, MAX and SUM

WHERE filters individual rows before grouping; HAVING filters the groups afterwards. That is why the minimum trip count is expressed with HAVING and not WHERE.

Subquery

Question: which zones are busier than the average zone? The inner query works out the average number of bookings per zone; the outer query keeps only the zones above it.

```
SELECT z.name AS zone_name, z.kind,
       COUNT(b.booking_id) AS bookings
FROM zones z
JOIN bookings b ON b.zone_id = z.zone_id
GROUP BY z.zone_id, z.name, z.kind
HAVING COUNT(b.booking_id) > (
    SELECT AVG(per_zone) FROM (
        SELECT COUNT(*) AS per_zone
        FROM bookings GROUP BY zone_id
    ) AS t )
ORDER BY bookings DESC
LIMIT 8;
```

JourneyMind: A MultiCommute Decision System

```

JourneyMind DBMS practical
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind> mysql -h localhost -P 3307 -u PES1UG23CS024_ADISHREE_GUPTA -p -D journeymind --prompt='\u@\d> '
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 1738
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

PES1UG23CS024_ADISHREE_GUPTA@journeymind> SELECT z.name, z.kind, COUNT(*) AS trips
-> FROM bookings b
-> JOIN zones z ON z.zone_id = b.zone_id
-> GROUP BY z.name, z.kind
-> HAVING COUNT(*) > (SELECT AVG(c) FROM
-> (SELECT COUNT(*) AS c FROM bookings
-> GROUP BY zone_id) AS per_zone)
-> ORDER BY trips DESC LIMIT 6;
+-----+-----+-----+
| name                                     | kind       | trips |
+-----+-----+-----+
| Wipro Campus, Doddakannelli (Sarjapur Road) | place      | 12765 |
| Bommanahalli                               | metro_station | 8504  |
| M.G. Road                                  | place      | 6901  |
| Koramangala 5th Block                      | place      | 6222  |
| Old Airport Road Gate                      | place      | 3981  |
+-----+-----+-----+
5 rows in set (0.60 sec)

PES1UG23CS024_ADISHREE_GUPTA@journeymind>

```

Figure 11: A nested subquery inside a HAVING clause

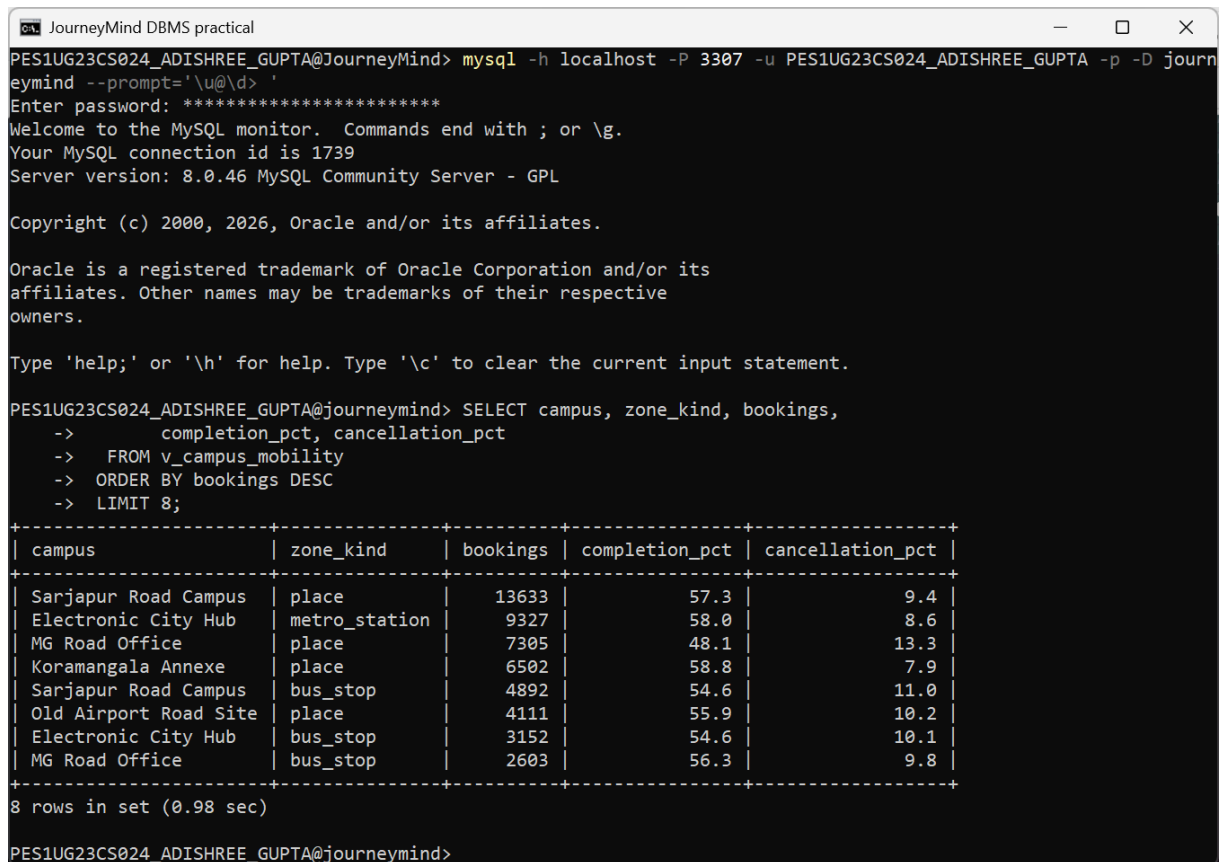
Querying the View

```

SELECT campus, zone_kind, bookings, completed_trips,
       completion_pct, cancellation_pct
FROM v_campus_mobility
ORDER BY bookings DESC
LIMIT 8;

```

JourneyMind: A MultiCommute Decision System



```

JourneyMind DBMS practical
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind> mysql -h localhost -P 3307 -u PES1UG23CS024_ADISHREE_GUPTA -p -D journeymind --prompt='\u@\d> '
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 1739
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

PES1UG23CS024_ADISHREE_GUPTA@journeymind> SELECT campus, zone_kind, bookings,
-> completion_pct, cancellation_pct
-> FROM v_campus_mobility
-> ORDER BY bookings DESC
-> LIMIT 8;
+-----+-----+-----+-----+-----+
| campus                | zone_kind | bookings | completion_pct | cancellation_pct |
+-----+-----+-----+-----+-----+
| Sarjapur Road Campus  | place     | 13633    | 57.3           | 9.4              |
| Electronic City Hub    | metro_station | 9327     | 58.0           | 8.6              |
| MG Road Office         | place     | 7305     | 48.1           | 13.3             |
| Koramangala Annexe     | place     | 6502     | 58.8           | 7.9              |
| Sarjapur Road Campus  | bus_stop  | 4892     | 54.6           | 11.0             |
| Old Airport Road Site  | place     | 4111     | 55.9           | 10.2             |
| Electronic City Hub    | bus_stop  | 3152     | 54.6           | 10.1             |
| MG Road Office         | bus_stop  | 2603     | 56.3           | 9.8              |
+-----+-----+-----+-----+-----+
8 rows in set (0.98 sec)

PES1UG23CS024_ADISHREE_GUPTA@journeymind>

```

Figure 12: Reading the v_campus_mobility view

Joining a Session to its Attempts

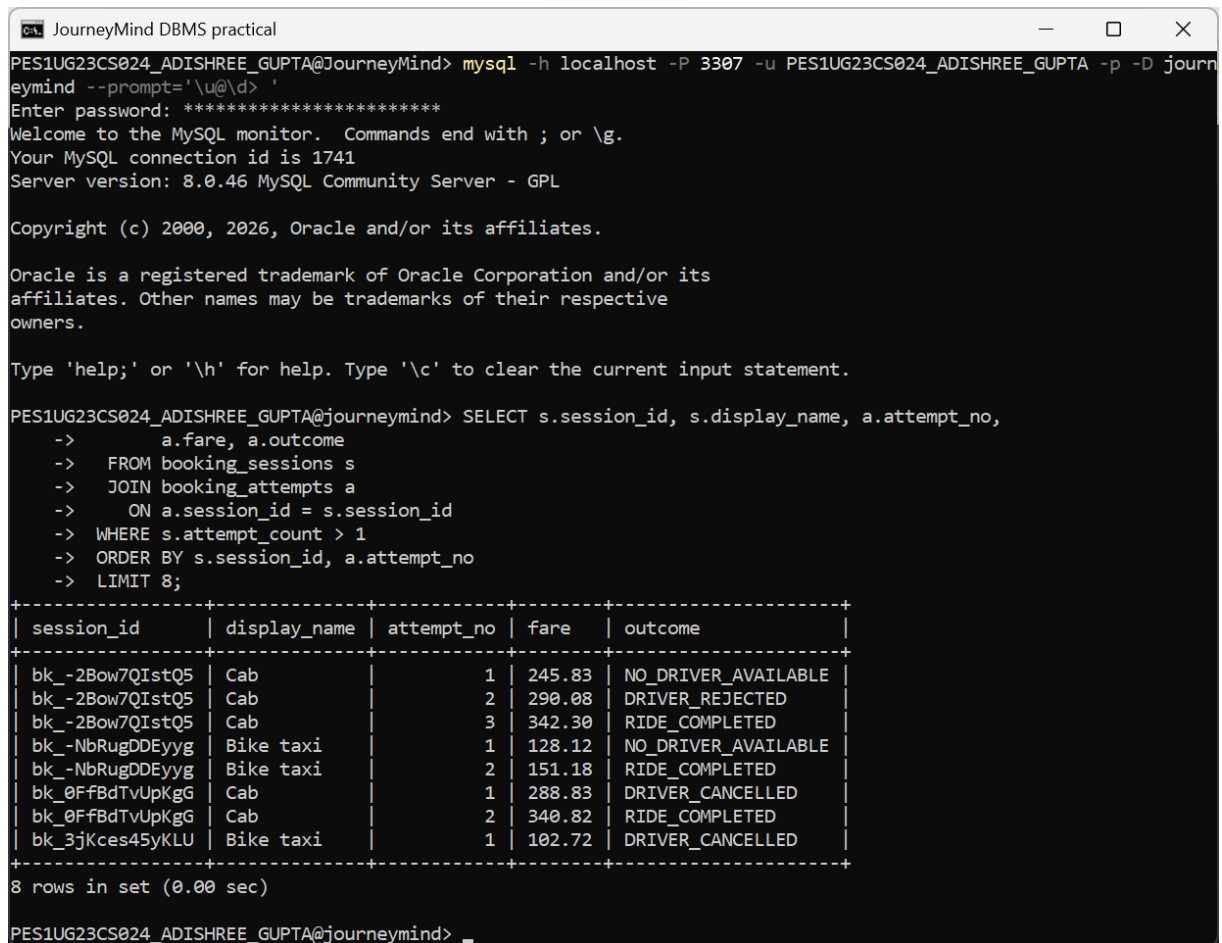
Question: for bookings that ran out of attempts, what happened on each try? This walks the one-to-many relationship and shows the fare rising with each retry.

```

SELECT s.session_id, s.display_name, a.attempt_no,
       a.fare, a.outcome, a.succeeded
FROM booking_sessions s
JOIN booking_attempts a ON a.session_id = s.session_id
WHERE s.exhausted = TRUE
ORDER BY s.created_at DESC, a.attempt_no
LIMIT 10;

```

JourneyMind: A MultiCommute Decision System



```

JourneyMind DBMS practical
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind> mysql -h localhost -P 3307 -u PES1UG23CS024_ADISHREE_GUPTA -p -D journeymind --prompt='\u@\d> '
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 1741
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

PES1UG23CS024_ADISHREE_GUPTA@journeymind> SELECT s.session_id, s.display_name, a.attempt_no,
->      a.fare, a.outcome
-> FROM booking_sessions s
-> JOIN booking_attempts a
->   ON a.session_id = s.session_id
-> WHERE s.attempt_count > 1
-> ORDER BY s.session_id, a.attempt_no
-> LIMIT 8;
+-----+-----+-----+-----+-----+
| session_id | display_name | attempt_no | fare | outcome |
+-----+-----+-----+-----+-----+
| bk_-2Bow7QIstQ5 | Cab | 1 | 245.83 | NO_DRIVER_AVAILABLE |
| bk_-2Bow7QIstQ5 | Cab | 2 | 290.08 | DRIVER_REJECTED |
| bk_-2Bow7QIstQ5 | Cab | 3 | 342.30 | RIDE_COMPLETED |
| bk_-NbRugDDEyyg | Bike taxi | 1 | 128.12 | NO_DRIVER_AVAILABLE |
| bk_-NbRugDDEyyg | Bike taxi | 2 | 151.18 | RIDE_COMPLETED |
| bk_0FfBdTvUpKgG | Cab | 1 | 288.83 | DRIVER_CANCELLED |
| bk_0FfBdTvUpKgG | Cab | 2 | 340.82 | RIDE_COMPLETED |
| bk_3jKces45yKLU | Bike taxi | 1 | 102.72 | DRIVER_CANCELLED |
+-----+-----+-----+-----+-----+
8 rows in set (0.00 sec)

PES1UG23CS024_ADISHREE_GUPTA@journeymind>

```

Figure 13: Following the one-to-many relationship from a booking session to its attempts

Transactions

When a booking finishes, the session row and all of its attempt rows describe one event. Writing the session and then failing to write the attempts would leave a booking in the database that appeared to have cost nothing. Both are therefore written inside a single transaction.

```

with mysql.transaction() as cur:
    cur.execute(INSERT_SESSION, (...))    # the booking itself
    cur.execute(DELETE_ATTEMPTS, (sid,))  # clear any previous attempts
    cur.executemany(INSERT_ATTEMPT, [...]) # every attempt that was made
# commit happens only if all three succeed; any error rolls back the lot

```

JourneyMind: A MultiCommute Decision System

```

C:\> JourneyMind DBMS practical
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind> mysql -h localhost -P 3307 -u PES1UG23CS024_ADISHREE_GUPTA -p -D journeymind --prompt='\u@\d> '
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 1759
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

PES1UG23CS024_ADISHREE_GUPTA@journeymind> SELECT COUNT(*) AS before_txn FROM demo_feedback;
+-----+
| before_txn |
+-----+
|          1 |
+-----+
1 row in set (0.00 sec)

PES1UG23CS024_ADISHREE_GUPTA@journeymind> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

PES1UG23CS024_ADISHREE_GUPTA@journeymind> INSERT INTO demo_feedback (session_id, rating)
-> VALUES ('bk_U0tEt9Hop9TN', 4);
Query OK, 1 row affected (0.00 sec)

PES1UG23CS024_ADISHREE_GUPTA@journeymind> INSERT INTO demo_feedback (session_id, rating)
-> VALUES ('bk_U0tEt9Hop9TN', 99);
ERROR 3819 (HY000): Check constraint 'chk_demo_rating' is violated.
PES1UG23CS024_ADISHREE_GUPTA@journeymind> ROLLBACK;
Query OK, 0 rows affected (0.00 sec)

PES1UG23CS024_ADISHREE_GUPTA@journeymind> SELECT COUNT(*) AS after_rollback FROM demo_feedback;
+-----+
| after_rollback |
+-----+
|          1 |
+-----+
1 row in set (0.00 sec)

PES1UG23CS024_ADISHREE_GUPTA@journeymind> DROP TABLE demo_feedback;
Query OK, 0 rows affected (0.02 sec)

PES1UG23CS024_ADISHREE_GUPTA@journeymind> 

```

Figure 14: A transaction rolling back: the valid row is undone because a later statement in the same transaction failed

NoSQL Operations: Redis

Redis is not queried with SQL. Data is read and written by key, using the command appropriate to the data type.

```

127.0.0.1:6380> GET jm:geo:cubbon park
[12.9782, 77.596, "Cubbon Park"]

127.0.0.1:6380> HGETALL jm:session:bk_dCpyeGunr0By
provider_id  bike_taxi
display_name Bike taxi
base_fare    215.0

```

JourneyMind: A MultiCommute Decision System

```

attempt_count 4
settled      0

127.0.0.1:6380> TTL jm:session:bk_dCpyeGunr0By
604800      (seconds remaining; -1 would mean never expires)

127.0.0.1:6380> LLEN jm:audit:recent
500        (bounded by LTRIM, so memory use stays constant)

```

NoSQL Operations: Neo4j Cypher

The query below is executed by the live application when the router builds its network, to decide which stops a hailed vehicle may use. `count(DISTINCT ...)` is aggregation performed inside the graph database.

```

// Executed by the live application when the router builds its graph
MATCH (s:Stop)
OPTIONAL MATCH (s)-[t:TRANSIT_LINK]-()
WITH s, count(DISTINCT t.route_id) AS routes_served
WHERE s.kind IN ['metro_station', 'place'] OR routes_served >= 2
RETURN s.stop_id AS stop_id
ORDER BY stop_id;

```

```

JourneyMind DBMS practical
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind> cypher-shell -a bolt://127.0.0.1:7688 -u PES1UG23CS024_ADISHREE_GUPTA
password:

Connected to Neo4j using Bolt protocol version 5.7 at bolt://127.0.0.1:7688 as user PES1UG23CS024_ADISHREE_GUPTA
.
Type :help for a list of available commands or :exit to exit the shell.
Note that Cypher queries must end with a semicolon.
PES1UG23CS024_ADISHREE_GUPTA@neo4j> MATCH (s:Stop) OPTIONAL MATCH (s)-[t:TRANSIT_LINK]-()
                                     WITH s, count(DISTINCT t.route_id) AS routes
                                     WHERE s.kind IN ['metro_station','place'] OR routes >= 2
                                     RETURN s.name AS name, s.kind AS kind, routes
                                     ORDER BY routes DESC, name LIMIT 8;

+-----+-----+-----+
| name                                | kind          | routes |
+-----+-----+-----+
| "Kanakapura Road stop 1"           | "bus_stop"    | 2      |
| "Kanakapura Road stop 5"           | "bus_stop"    | 2      |
| "Kanakapura Road stop 6"           | "bus_stop"    | 2      |
| "Kanakapura Road stop 8"           | "bus_stop"    | 2      |
| "MG Road corridor stop 10"         | "bus_stop"    | 2      |
| "Majestic (Kempegowda)"            | "metro_station" | 2      |
| "Rashtreeya Vidyalaya Road"         | "metro_station" | 2      |
| "BTM Layout"                       | "metro_station" | 1      |
+-----+-----+-----+

8 rows
ready to start consuming query after 1 ms, results consumed after another 3 ms
PES1UG23CS024_ADISHREE_GUPTA@neo4j>

```

Figure 15: The ride-hub query executed by the running application

The next query is used for verification and demonstration and is not called during ordinary use. It is included because it shows most clearly why a graph database suits this data: the depth of the search is simply a number in the pattern.

```

// Demonstration / verification query - not called during ordinary use
MATCH (a:Stop {stop_id: $from_id}), (b:Stop {stop_id: $to_id})

```

JourneyMind: A MultiCommute Decision System

```
MATCH p = shortestPath((a)-[:TRANSIT_LINK|TRANSFER_LINK*..15]-(b))
RETURN length(p) AS hops, [n IN nodes(p) | n.name] AS stops_on_the_way;
```

```
JourneyMind DBMS practical
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind> cypher-shell -a bolt://127.0.0.1:7688 -u PES1UG23CS024_ADISHREE_GUPTA
password:

Connected to Neo4j using Bolt protocol version 5.7 at bolt://127.0.0.1:7688 as user PES1UG23CS024_ADISHREE_GUPTA
.
Type :help for a list of available commands or :exit to exit the shell.
Note that Cypher queries must end with a semicolon.
PES1UG23CS024_ADISHREE_GUPTA@neo4j> MATCH (a:Stop {stop_id: 'mg_majestic'}),
                                     (b:Stop {stop_id: 'mg_silkboard'})
                                     MATCH p = shortestPath(
                                     (a)-[:TRANSIT_LINK|TRANSFER_LINK*..15]-(b))
                                     RETURN length(p) AS hops,
                                     [n IN nodes(p) | n.name] AS stops_on_the_way;

+-----+
| hops | stops_on_the_way |
+-----+
| 11   | ["Majestic (Kempegowda)", "Chickpete", "Krishna Rajendra Market", "National College", "Lalbagh", "South End Circle", "Jayanagar", "Rashtriya Vidyalaya Road", "Ragigudda", "Jayadeva Hospital", "BTM Layout", "Central Silk Board"] |
+-----+

1 row
ready to start consuming query after 32 ms, results consumed after another 2 ms
PES1UG23CS024_ADISHREE_GUPTA@neo4j>
```

Figure 16: Graph traversal finding the shortest path between two stops

SQL DCL Statements

DCL (Data Control Language) statements control who may do what. JourneyMind applies access control at the database level and again at the application level.

Database-Level Access Control

```
-- Executed once by scripts/databases.ps1 when the database is created.
-- The application never connects as root; it uses a dedicated account whose
-- privileges are limited to the journeymind database only.

CREATE DATABASE IF NOT EXISTS journeymind
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

CREATE USER IF NOT EXISTS 'journeymind'@'127.0.0.1'
  IDENTIFIED BY '<password supplied from .env, never hard-coded>';

GRANT ALL PRIVILEGES ON journeymind.* TO 'journeymind'@'127.0.0.1';

FLUSH PRIVILEGES;
```

Access control is applied at two levels. At the database level the statements above create a dedicated MySQL account whose privileges are restricted to the journeymind database; the application never connects as root, and the password is supplied through an environment variable rather than written into the source code.

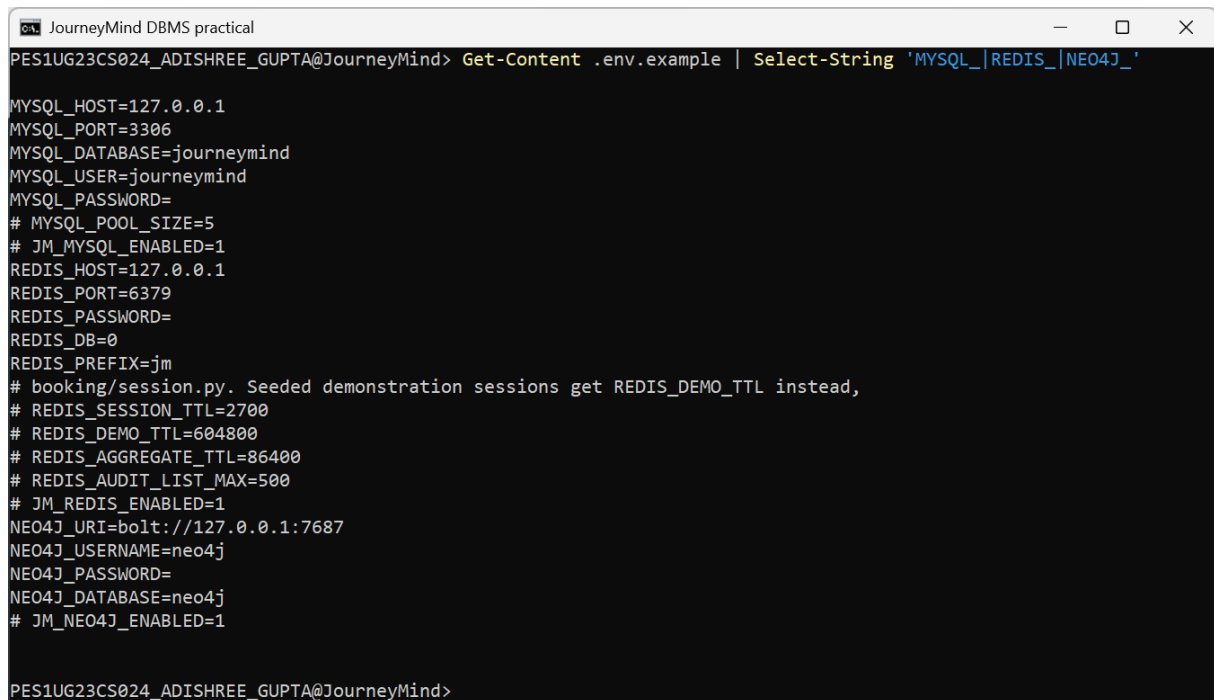
At the application level, FastAPI enforces role-based authorisation. Requests to the enterprise endpoints must carry an X-API-Key header. Keys are never stored in plain text: only their SHA-256 hash is kept, and the supplied key is hashed and compared. Three roles are defined, RIDER, ANALYST and ADMIN, and an endpoint declares the minimum role it requires.

Application-Level Authorisation

Role	Level	May access
RIDER	10	Journey search, comparison and booking
ANALYST	20	All of the above, plus the enterprise dashboard and the audit trail
ADMIN	30	All of the above; intended for configuration and key management

Table 10: Application roles enforced by the API

An endpoint declares the minimum role it requires. If no keys are configured and demonstration mode is switched off, the enterprise endpoints refuse every request rather than falling open.

JourneyMind: A MultiCommute Decision System

```
JourneyMind DBMS practical
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind> Get-Content .env.example | Select-String 'MYSQL_|REDIS_|NEO4J_|'

MYSQL_HOST=127.0.0.1
MYSQL_PORT=3306
MYSQL_DATABASE=journeymind
MYSQL_USER=journeymind
MYSQL_PASSWORD=
# MYSQL_POOL_SIZE=5
# JM_MYSQL_ENABLED=1
REDIS_HOST=127.0.0.1
REDIS_PORT=6379
REDIS_PASSWORD=
REDIS_DB=0
REDIS_PREFIX=jm
# booking/session.py. Seeded demonstration sessions get REDIS_DEMO_TTL instead,
# REDIS_SESSION_TTL=2700
# REDIS_DEMO_TTL=604800
# REDIS_AGGREGATE_TTL=86400
# REDIS_AUDIT_LIST_MAX=500
# JM_REDIS_ENABLED=1
NEO4J_URI=bolt://127.0.0.1:7687
NEO4J_USERNAME=neo4j
NEO4J_PASSWORD=
NEO4J_DATABASE=neo4j
# JM_NEO4J_ENABLED=1

PES1UG23CS024_ADISHREE_GUPTA@JourneyMind>
```

Figure 17: Database settings are supplied through environment variables; the committed template contains placeholders only

Results / Resulting Tables – Screenshots

Database Initialisation

All three databases are created and populated by a single documented command, which then runs the verification script as a separate process:

```
python database\init_all.py
```

```

JourneyMind DBMS practical
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind> .venv\Scripts\python.exe database\init_all.py --help | Select-Object -First 10
usage: init_all.py [-h] [--warm] [--reset] [--trips TRIPS] [--skip-verify]

One command that takes three empty databases to a verified demonstration state.

python database/init_all.py          # full initialisation, then verify
python database/init_all.py --warm   # only restore the expiring Redis
                                     # families, then verify
python database/init_all.py --reset  # drop the MySQL tables first

WHY THIS EXISTS
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind> .venv\Scripts\python.exe database\init_all.py --help | Select-Object -Last 14
-----
`jm:session:*` and `jm:agg:*` carry TTLs because the data genuinely is
short-lived -- a booking session is worthless an hour later and a cached
aggregate goes stale. That is correct Redis behaviour and it is not changed
here. It does mean the counts fall over time, so `--warm` re-runs step 4 and
brings both families back above 100 in about a minute, without touching MySQL,
Neo4j or the geocode cache.

options:
-h, --help      show this help message and exit
--warm          only restore the TTL-backed Redis families, then verify
--reset         drop the MySQL tables before reloading them
--trips TRIPS   how many demonstration bookings to run (default 130)
--skip-verify   seed only; do not run database/verify.py afterwards
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind>

```

Figure 18: The single command that initialises all three databases and then verifies them

```

JourneyMind DBMS practical
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind> powershell -ExecutionPolicy Bypass -File scripts\databases.ps1 -Action status

JourneyMind databases - status
-----

Ports
MySQL      listening on 3307
Redis      listening on 6380
Neo4j (bolt) listening on 7688

What the application can actually reach
ok  mysql
ok  redis
ok  neo4j

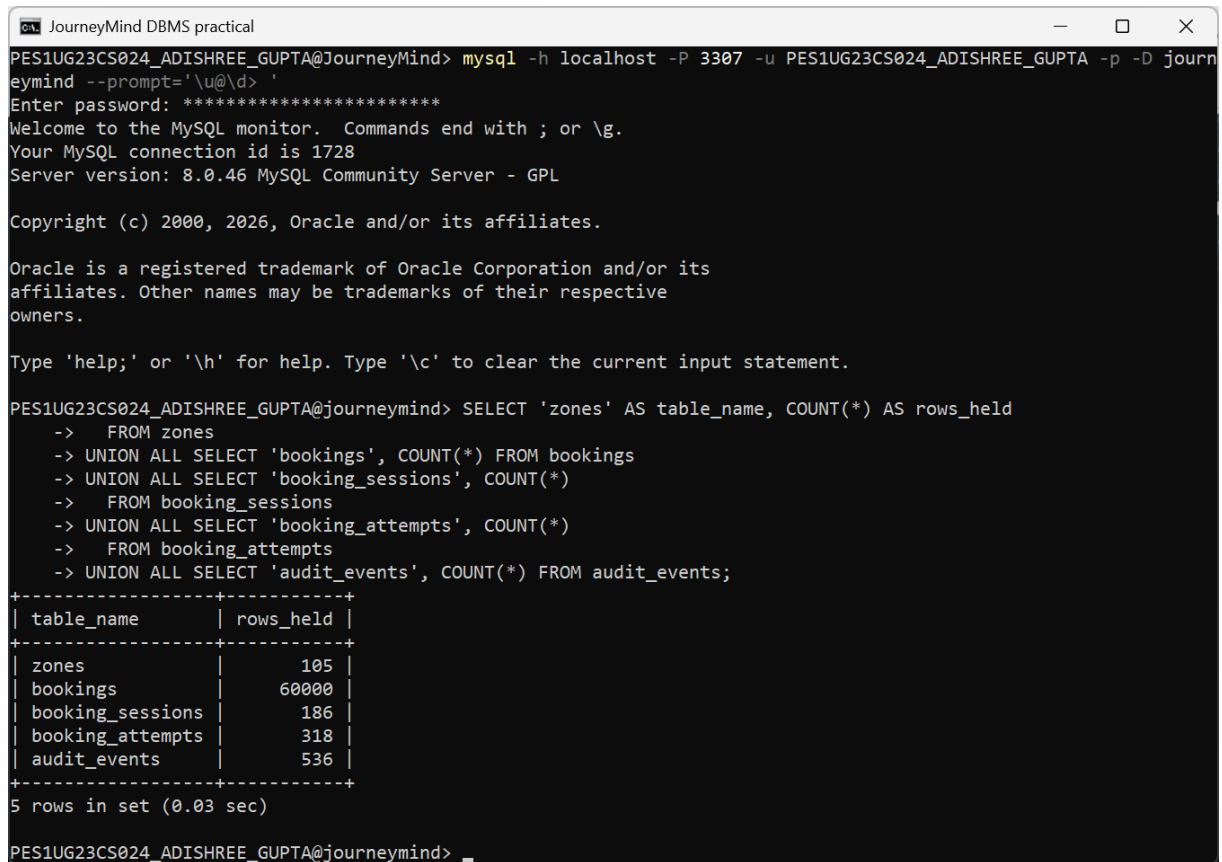
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind>

```

Figure 19: MySQL, Redis and Neo4j running and reachable by the application

JourneyMind: A MultiCommute Decision System

Resulting Tables in MySQL



```

JourneyMind DBMS practical
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind> mysql -h localhost -P 3307 -u PES1UG23CS024_ADISHREE_GUPTA -p -D journeymind --prompt='\u@\d> '
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 1728
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

PES1UG23CS024_ADISHREE_GUPTA@journeymind> SELECT 'zones' AS table_name, COUNT(*) AS rows_held
-> FROM zones
-> UNION ALL SELECT 'bookings', COUNT(*) FROM bookings
-> UNION ALL SELECT 'booking_sessions', COUNT(*)
-> FROM booking_sessions
-> UNION ALL SELECT 'booking_attempts', COUNT(*)
-> FROM booking_attempts
-> UNION ALL SELECT 'audit_events', COUNT(*) FROM audit_events;
+-----+-----+
| table_name | rows_held |
+-----+-----+
| zones      | 105       |
| bookings   | 60000     |
| booking_sessions | 186       |
| booking_attempts | 318       |
| audit_events | 536       |
+-----+-----+
5 rows in set (0.03 sec)

PES1UG23CS024_ADISHREE_GUPTA@journeymind>

```

Figure 20: Row counts for every table after initialisation

JourneyMind: A MultiCommute Decision System

```

JourneyMind DBMS practical
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind> mysql -h localhost -P 3307 -u PES1UG23CS024_ADISHREE_GUPTA -p -D journeymind --prompt='\u@>'
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 1723
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

PES1UG23CS024_ADISHREE_GUPTA@journeymind> DESCRIBE booking_sessions;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| session_id | varchar(32) | NO | PRI | NULL |  |
| created_at | datetime | NO | MUL | NULL |  |
| departure | datetime | NO |  | NULL |  |
| provider_id | varchar(32) | NO | MUL | NULL |  |
| display_name | varchar(64) | NO |  | NULL |  |
| mode | varchar(24) | NO |  | NULL |  |
| service_class | varchar(24) | NO |  | NULL |  |
| origin_label | varchar(160) | NO |  | NULL |  |
| dest_label | varchar(160) | NO |  | NULL |  |
| advertised_fare | decimal(10,2) | NO |  | NULL |  |
| paid_fare | decimal(10,2) | YES |  | NULL |  |
| p_match | decimal(6,5) | NO |  | NULL |  |
| p_accept | decimal(6,5) | NO |  | NULL |  |
| p_cancel | decimal(6,5) | NO |  | NULL |  |
| attempt_count | tinyint | NO |  | 0 |  |
| wasted_min | decimal(7,2) | NO |  | 0.00 |  |
| settled | tinyint(1) | NO |  | 0 |  |
| exhausted | tinyint(1) | NO |  | 0 |  |
| demo | tinyint(1) | NO |  | 0 |  |
+-----+-----+-----+-----+-----+-----+
19 rows in set (0.01 sec)

PES1UG23CS024_ADISHREE_GUPTA@journeymind>

```

Figure 21: Structure of the booking_sessions table, read from information_schema

Data Volume Verification

The project requirement is that every database structure holds at least 100 meaningful records. The counts below are read from the running databases by database/verify.py immediately after a clean initialisation; none is stored or assumed. The script returns exit code 0 only when every structure passes.

Database	Structure	Count	Required	Status
MySQL	zones	105	100	PASS
MySQL	bookings	60,000	100	PASS
MySQL	booking_sessions	130	100	PASS
MySQL	booking_attempts	227	100	PASS
MySQL	audit_events	371	100	PASS
Redis	cached place lookups	222	100	PASS
Redis	live booking sessions	130	100	PASS
Redis	cached dashboard payloads	111	100	PASS
Redis	recent decisions list	371	100	PASS
Neo4j	(:Stop) nodes	223	100	PASS

JourneyMind: A MultiCommute Decision System

Neo4j	[:ROAD_LINK]	1,052	100	PASS
Neo4j	[:TRANSIT_LINK]	174	100	PASS
Neo4j	[:TRANSFER_LINK]	216	100	PASS

Table 11: Database record verification after clean initialisation (all structures pass)

```

PES1UG23CS024_ADISHREE_GUPTA@JourneyMind> .venv\Scripts\python.exe database\verify.py

=====
DATABASE DATA VOLUME VERIFICATION
=====
Counts below are read from the running databases at this moment.

referential integrity: 0 bookings with no zone, 0 attempts with no session (both zero, as the foreign keys require)

TTL sample: jm:session:bk_OKK3dvX0qb-1 expires in 595850 s

disconnected (:Stop) nodes: 0 (none – every node is on the network)
sample traversal (Majestic -> Silk Board): 11 hops, Majestic (Kempegowda) -> Chickpete -> Krishna Rajendra Mar
ket -> National College -> ...

| Technology | Structure | Count | Required | Result |
|-----|-----|-----|-----|-----|
| MySQL | zones | 105 | >=100 | PASS |
| MySQL | bookings | 60,000 | >=100 | PASS |
| MySQL | booking_sessions | 186 | >=100 | PASS |
| MySQL | booking_attempts | 318 | >=100 | PASS |
| MySQL | audit_events | 536 | >=100 | PASS |
| Redis | cached place lookups | 222 | >=100 | PASS |
| Redis | live booking sessions | 163 | >=100 | PASS |
| Redis | cached dashboard payloads | 112 | >=100 | PASS |
| Redis | recent decisions list | 500 | >=100 | PASS |
| Neo4j | (:Stop) nodes | 223 | >=100 | PASS |
| Neo4j | [:ROAD_LINK] | 1,052 | >=100 | PASS |
| Neo4j | [:TRANSIT_LINK] | 174 | >=100 | PASS |
| Neo4j | [:TRANSFER_LINK] | 216 | >=100 | PASS |

zones: pickup/drop zones the booking history refers to
bookings: the enterprise booking history (fact table)
booking_sessions: bookings this instance made, one per BOOK NOW
booking_attempts: each attempt inside a session
audit_events: every AI decision and the evidence behind it
cached place lookups: string keys matching jm:geo:*
live booking sessions: hash keys matching jm:session:* – TTL-backed (REDIS_DEMO_TTL, 7 days for seeded session
s); restore with `python database/init_all.py --warm`
cached dashboard payloads: string keys matching jm:agg:* – TTL-backed (REDIS_AGGREGATE_TTL, 1 day); restore wi
th `python database/init_all.py --warm`
recent decisions list: list entries in jm:audit:recent (LLEN, bounded by LTRIM)
(:Stop) nodes: every vertex the router can stand on
[:ROAD_LINK]: walkable street segments, both directions
[:TRANSIT_LINK]: stop-to-stop hops on metro/bus routes
[:TRANSFER_LINK]: walks between nearby transit nodes

All 13 structures meet the 100+ requirement.
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind>

```

Figure 22: Verification output read from the live databases

Integrity Checks

The same verification also confirms that the data is meaningful rather than merely numerous:

JourneyMind: A MultiCommute Decision System

- **0 bookings** reference a zone that does not exist, and **0 attempts** reference a missing session — the foreign keys hold.
- **0 disconnected Stop nodes** — every node in the graph takes part in the network.
- A sample traversal from Majestic to Central Silk Board returns a real 11-hop path.

Redis Structures

```

JourneyMind DBMS practical
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind-Redis> $conn = "-p","6380","--user","PES1UG23CS024_ADISHREE_GUPTA"
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind-Redis> redis-cli @conn ACL WHOAMI
"PES1UG23CS024_ADISHREE_GUPTA"
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind-Redis> redis-cli @conn DBSIZE
(integer) 498
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind-Redis> redis-cli @conn SCAN 0 MATCH jm:geo:* COUNT 6
1) "448"
2) 1) "jm:geo:junction 099"
   2) "jm:geo:junction 023"
   3) "jm:geo:sarjapur road stop 1"
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind-Redis> redis-cli @conn TYPE jm:audit:recent
list
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind-Redis> redis-cli @conn LLEN jm:audit:recent
(integer) 500
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind-Redis>

```

Figure 23: The four Redis key families used by JourneyMind

```

JourneyMind DBMS practical
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind-Redis> $conn = "-p","6380","--user","PES1UG23CS024_ADISHREE_GUPTA"
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind-Redis> redis-cli @conn ACL WHOAMI
"PES1UG23CS024_ADISHREE_GUPTA"
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind-Redis> redis-cli @conn TYPE jm:session:bk_OKK3dvX0qb-1
hash
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind-Redis> redis-cli @conn HLEN jm:session:bk_OKK3dvX0qb-1
(integer) 22
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind-Redis> redis-cli @conn HGET jm:session:bk_OKK3dvX0qb-1 display_name
"Bike taxi"
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind-Redis> redis-cli @conn HGET jm:session:bk_OKK3dvX0qb-1 origin_label
"Lalbagh West Gate"
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind-Redis> redis-cli @conn HGET jm:session:bk_OKK3dvX0qb-1 dest_label
"M.G. Road"
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind-Redis> redis-cli @conn TTL jm:session:bk_OKK3dvX0qb-1
(integer) 595334
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind-Redis>

```

Figure 24: A live booking session stored as a Redis HASH

```

JourneyMind DBMS practical
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind-Redis> $conn = "-p","6380","--user","PES1UG23CS024_ADISHREE_GUPTA"
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind-Redis> redis-cli @conn ACL WHOAMI
"PES1UG23CS024_ADISHREE_GUPTA"
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind-Redis> redis-cli @conn TTL jm:session:bk_OKK3dvX0qb-1
(integer) 595353
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind-Redis> redis-cli @conn TTL "jm:geo:junction 099"
(integer) -1
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind-Redis> redis-cli @conn INFO keypace
# Keypace
db0:keys=498,expires=275,avg_ttl=384931000,subexpiry=0
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind-Redis>

```

Figure 25: Time-to-live values: cached places never expire, sessions and cached dashboard answers do

JourneyMind: A MultiCommute Decision System

Two of the four families expire on purpose. That is correct behaviour for a session store and a cache rather than data loss, and nothing in MySQL is affected. The command `python database\init_all.py --warm` restores them at any time.

Neo4j Graph

```

JourneyMind DBMS practical
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind> cypher-shell -a bolt://127.0.0.1:7688 -u PES1UG23CS024_ADISHREE_GUPTA
password:

Connected to Neo4j using Bolt protocol version 5.7 at bolt://127.0.0.1:7688 as user PES1UG23CS024_ADISHREE_GUPTA
.
Type :help for a list of available commands or :exit to exit the shell.
Note that Cypher queries must end with a semicolon.
PES1UG23CS024_ADISHREE_GUPTA@neo4j> SHOW CURRENT USER;
+-----+
| user | roles | passwordChangeRequired | suspended | home |
+-----+
| "PES1UG23CS024_ADISHREE_GUPTA" | NULL | FALSE | NULL | NULL |
+-----+

1 row
ready to start consuming query after 1 ms, results consumed after another 0 ms
PES1UG23CS024_ADISHREE_GUPTA@neo4j> MATCH (n:Stop) RETURN count(n) AS stop_nodes;
+-----+
| stop_nodes |
+-----+
| 223 |
+-----+

1 row
ready to start consuming query after 0 ms, results consumed after another 0 ms
PES1UG23CS024_ADISHREE_GUPTA@neo4j> MATCH ()-[r:ROAD_LINK]->() RETURN count(r) AS road_links;
+-----+
| road_links |
+-----+
| 1052 |
+-----+

1 row
ready to start consuming query after 1 ms, results consumed after another 0 ms
PES1UG23CS024_ADISHREE_GUPTA@neo4j> MATCH ()-[r:TRANSIT_LINK]->() RETURN count(r) AS transit_links;
+-----+
| transit_links |
+-----+
| 174 |
+-----+

1 row
ready to start consuming query after 0 ms, results consumed after another 0 ms
PES1UG23CS024_ADISHREE_GUPTA@neo4j> MATCH ()-[r:TRANSFER_LINK]->() RETURN count(r) AS transfer_links;
+-----+
| transfer_links |
+-----+
| 216 |
+-----+

1 row
ready to start consuming query after 1 ms, results consumed after another 1 ms
PES1UG23CS024_ADISHREE_GUPTA@neo4j>

```

Figure 26: Node labels and relationship types present in the graph

JourneyMind: A MultiCommute Decision System

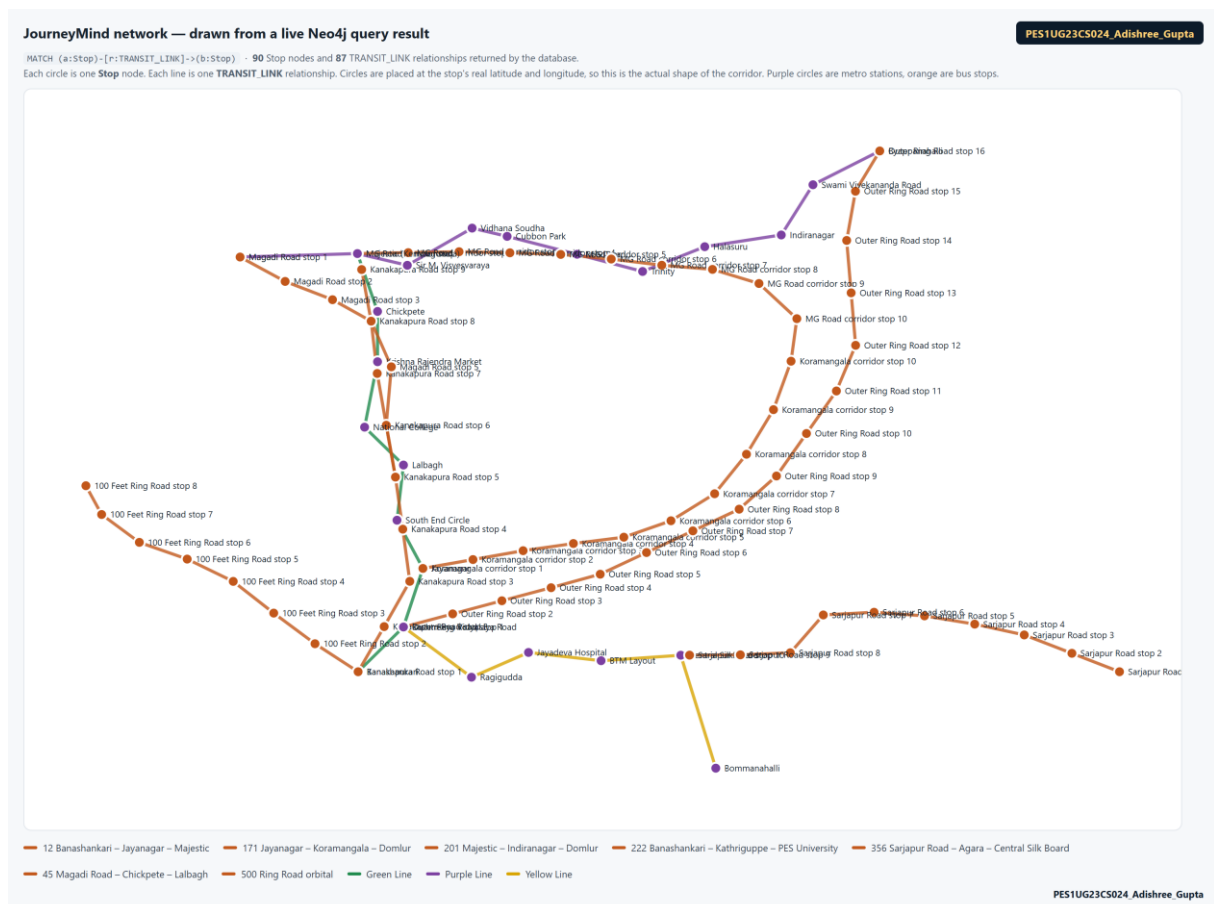
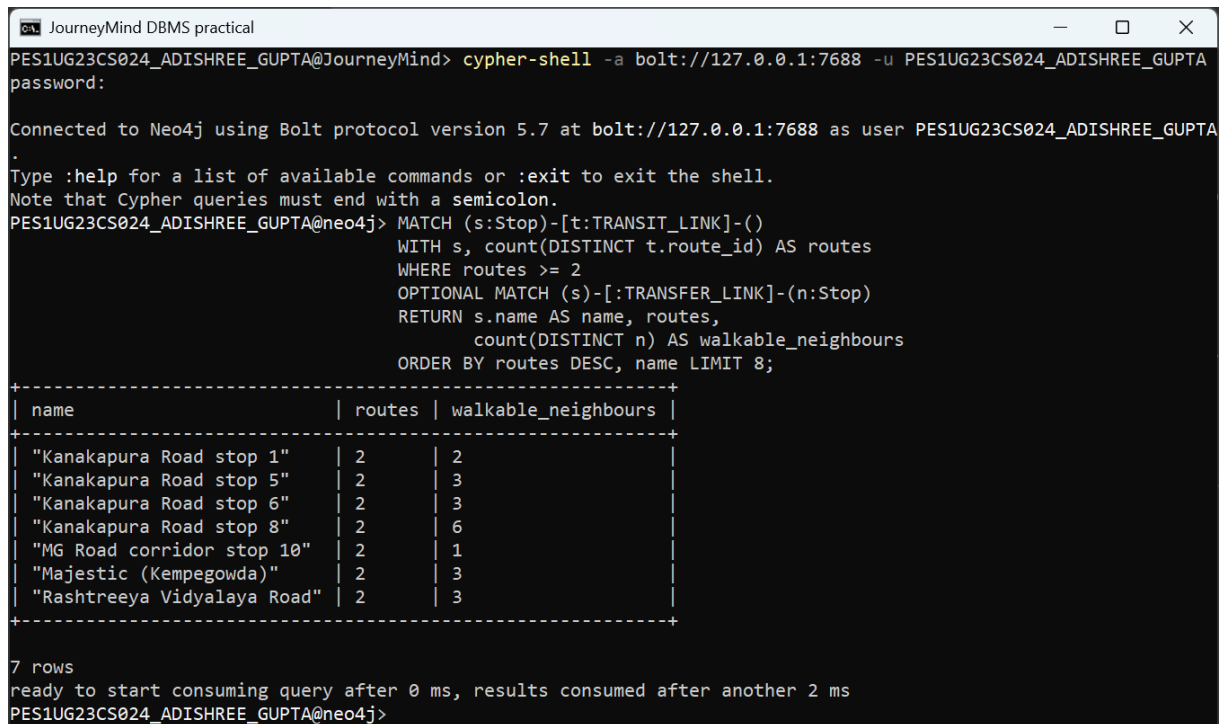


Figure 27: The transport network drawn from a live Neo4j query result; each circle is a Stop node and each line a TRANSIT_LINK relationship

JourneyMind: A MultiCommute Decision System



```

JourneyMind DBMS practical
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind> cypher-shell -a bolt://127.0.0.1:7688 -u PES1UG23CS024_ADISHREE_GUPTA
password:

Connected to Neo4j using Bolt protocol version 5.7 at bolt://127.0.0.1:7688 as user PES1UG23CS024_ADISHREE_GUPTA
.
Type :help for a list of available commands or :exit to exit the shell.
Note that Cypher queries must end with a semicolon.
PES1UG23CS024_ADISHREE_GUPTA@neo4j> MATCH (s:Stop)-[t:TRANSIT_LINK]-()
    WITH s, count(DISTINCT t.route_id) AS routes
    WHERE routes >= 2
    OPTIONAL MATCH (s)-[:TRANSFER_LINK]-(n:Stop)
    RETURN s.name AS name, routes,
           count(DISTINCT n) AS walkable_neighbours
    ORDER BY routes DESC, name LIMIT 8;
+-----+-----+-----+
| name                                | routes | walkable_neighbours |
+-----+-----+-----+
| "Kanakapura Road stop 1"           | 2      | 2                   |
| "Kanakapura Road stop 5"           | 2      | 3                   |
| "Kanakapura Road stop 6"           | 2      | 3                   |
| "Kanakapura Road stop 8"           | 2      | 6                   |
| "MG Road corridor stop 10"         | 2      | 1                   |
| "Majestic (Kempegowda)"           | 2      | 3                   |
| "Rashtreeya Vidyalaya Road"        | 2      | 3                   |
+-----+-----+-----+

7 rows
ready to start consuming query after 0 ms, results consumed after another 2 ms
PES1UG23CS024_ADISHREE_GUPTA@neo4j>

```

Figure 28: Aggregation inside Neo4j: the stops where the most routes meet

Database and Application Integration

The figure below records the database counts before and after one real press of BOOK NOW, and then reads back the exact MySQL row that the booking created. It demonstrates that the databases are genuinely part of the application rather than configured alongside it.

JourneyMind: A MultiCommute Decision System

```

PES1UG23CS024_ADISHREE_GUPTA@JourneyMind> .venv\Scripts\python.exe scripts\integration_proof.py
BEFORE the booking
MySQL booking_sessions : 186
MySQL booking_attempts : 318
MySQL audit_events : 536
Redis jm:session:* : 163
Redis jm:audit:recent : 500

User presses BOOK NOW on: bike_taxi
session bk_ANZ1xCvom_GZ attempts=4 settled=False

AFTER the booking
MySQL booking_sessions : 187 (+1)
MySQL booking_attempts : 322 (+4)
MySQL audit_events : 538 (+2)
Redis jm:session:* : 164 (+1)
Redis jm:audit:recent : 500 (+0)

The new MySQL row for that exact booking:
session_id      bk_ANZ1xCvom_GZ
display_name     Bike taxi
advertised_fare  71.83
attempt_count    4
settled          0
exhausted        1
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind>

```

Figure 29: One real booking, showing the MySQL and Redis counts before and after, and the resulting database row

Testing

The project is covered by 316 automated tests. With all three databases running, 315 pass and 1 is skipped. The skip is expected and documented: one test requires a scheduled bus or metro service on a particular trip and skips when none is running at that hour.

Test layer	Command	Result
Unit and API tests	pytest tests -q -m "not integration"	289 passed
Database integration	pytest tests/integration -v	26 passed
Data volume verification	python database/verify.py	13 of 13 structures PASS (exit code 0)
Full suite	pytest tests -q	315 passed, 1 skipped

Table 12: Test results

```

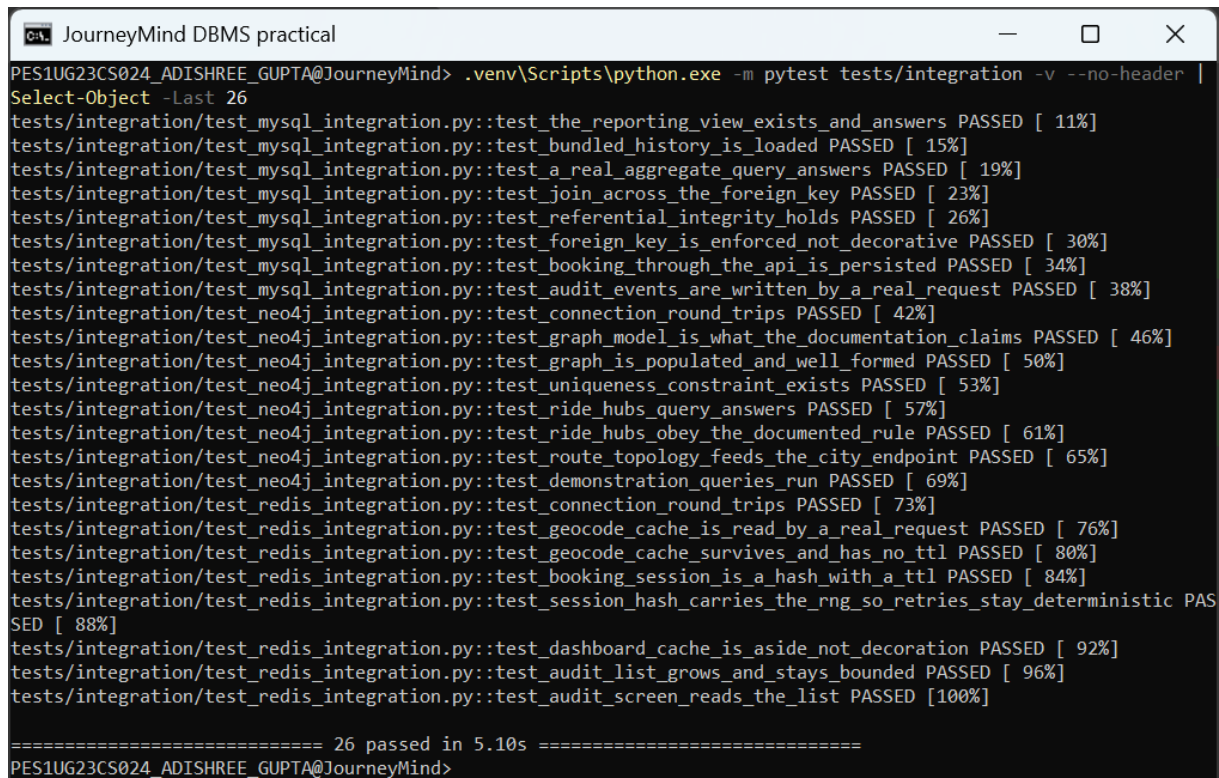
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind> .venv\Scripts\python.exe -m pytest tests -q | Select-Object -Last 12
.....s..... [ 22%]
..... [ 45%]
..... [ 68%]
..... [ 91%]
..... [100%]
===== warnings summary =====
tests/test_model_parity.py::test_numpy_matches_pytorch[gat]
  C:\Users\abcom\OneDrive\Desktop\JourneyMind\tests\..\backend\app\models\gnn_torch.py:58: UserWarning: index_reduce() is in beta and the API may change at any time. (Triggered internally at C:\actions-runner\work\pytorch\pytorch\builder\windows\pytorch\aten\src\ATen\native\TensorAdvancedIndexing.cpp:1222.)
    max_per = max_per.index_reduce(0, index, logits, "amax", include_self=True)

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
315 passed, 1 skipped, 1 warning in 363.21s (0:06:03)
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind>

```

Figure 30: Automated test suite results

JourneyMind: A MultiCommute Decision System



```

JourneyMind DBMS practical
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind> .venv\Scripts\python.exe -m pytest tests/integration -v --no-header |
Select-Object -Last 26
tests/integration/test_mysql_integration.py::test_the_reporting_view_exists_and_answers PASSED [ 11%]
tests/integration/test_mysql_integration.py::test_bundled_history_is_loaded PASSED [ 15%]
tests/integration/test_mysql_integration.py::test_a_real_aggregate_query_answers PASSED [ 19%]
tests/integration/test_mysql_integration.py::test_join_across_the_foreign_key PASSED [ 23%]
tests/integration/test_mysql_integration.py::test_referential_integrity_holds PASSED [ 26%]
tests/integration/test_mysql_integration.py::test_foreign_key_is_enforced_not_decorative PASSED [ 30%]
tests/integration/test_mysql_integration.py::test_booking_through_the_api_is_persisted PASSED [ 34%]
tests/integration/test_mysql_integration.py::test_audit_events_are_written_by_a_real_request PASSED [ 38%]
tests/integration/test_neo4j_integration.py::test_connection_round_trips PASSED [ 42%]
tests/integration/test_neo4j_integration.py::test_graph_model_is_what_the_documentation_claims PASSED [ 46%]
tests/integration/test_neo4j_integration.py::test_graph_is_populated_and_well_formed PASSED [ 50%]
tests/integration/test_neo4j_integration.py::test_uniqueness_constraint_exists PASSED [ 53%]
tests/integration/test_neo4j_integration.py::test_ride_hubs_query_answers PASSED [ 57%]
tests/integration/test_neo4j_integration.py::test_ride_hubs_obey_the_documented_rule PASSED [ 61%]
tests/integration/test_neo4j_integration.py::test_route_topology_feeds_the_city_endpoint PASSED [ 65%]
tests/integration/test_neo4j_integration.py::test_demonstration_queries_run PASSED [ 69%]
tests/integration/test_redis_integration.py::test_connection_round_trips PASSED [ 73%]
tests/integration/test_redis_integration.py::test_geocode_cache_is_read_by_a_real_request PASSED [ 76%]
tests/integration/test_redis_integration.py::test_geocode_cache_survives_and_has_no_ttl PASSED [ 80%]
tests/integration/test_redis_integration.py::test_booking_session_is_a_hash_with_a_ttl PASSED [ 84%]
tests/integration/test_redis_integration.py::test_session_hash_carries_the_rng_so_retries_stay_deterministic PASSED [ 88%]
tests/integration/test_redis_integration.py::test_dashboard_cache_is_aside_not_decoration PASSED [ 92%]
tests/integration/test_redis_integration.py::test_audit_list_grows_and_stays_bounded PASSED [ 96%]
tests/integration/test_redis_integration.py::test_audit_screen_reads_the_list PASSED [100%]

===== 26 passed in 5.10s =====
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind>

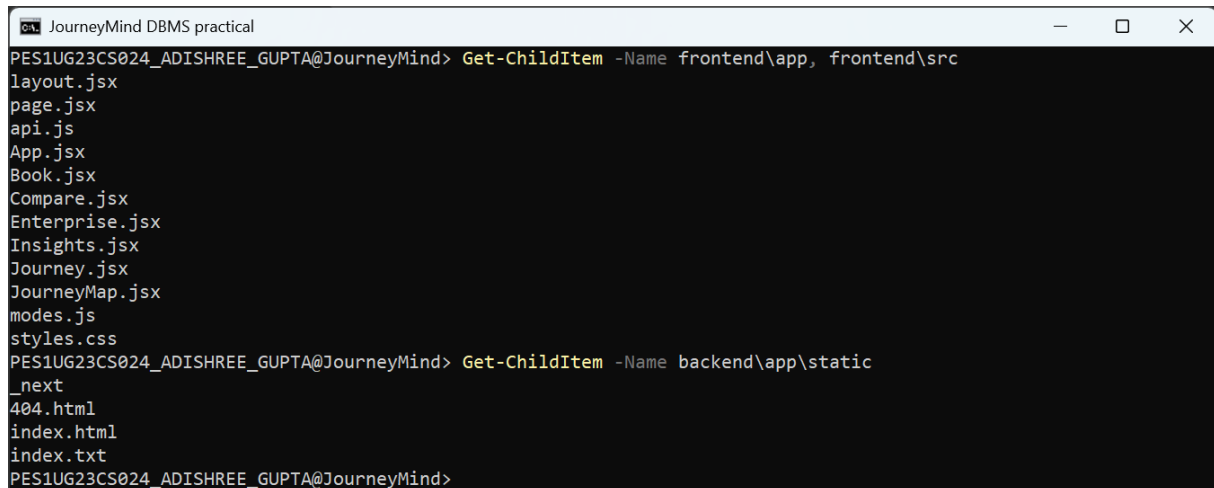
```

Figure 31: Database integration tests, which require all three databases to be running

The integration tests do more than open a connection. One temporarily replaces a value in Redis and checks that the API's answer follows it, proving the application really reads Redis; another inserts a deliberately invalid row to confirm the foreign key refuses it.

Application Front-end Screenshots

The front end is built with Next.js 14 using the App Router. The two entry files are `frontend/app/layout.jsx`, which provides the page shell, and `frontend/app/page.jsx`, which mounts the application. Build settings are in `frontend/next.config.mjs`. Running `npm run build` produces static files which the FastAPI back end then serves, so the whole product runs as a single service.

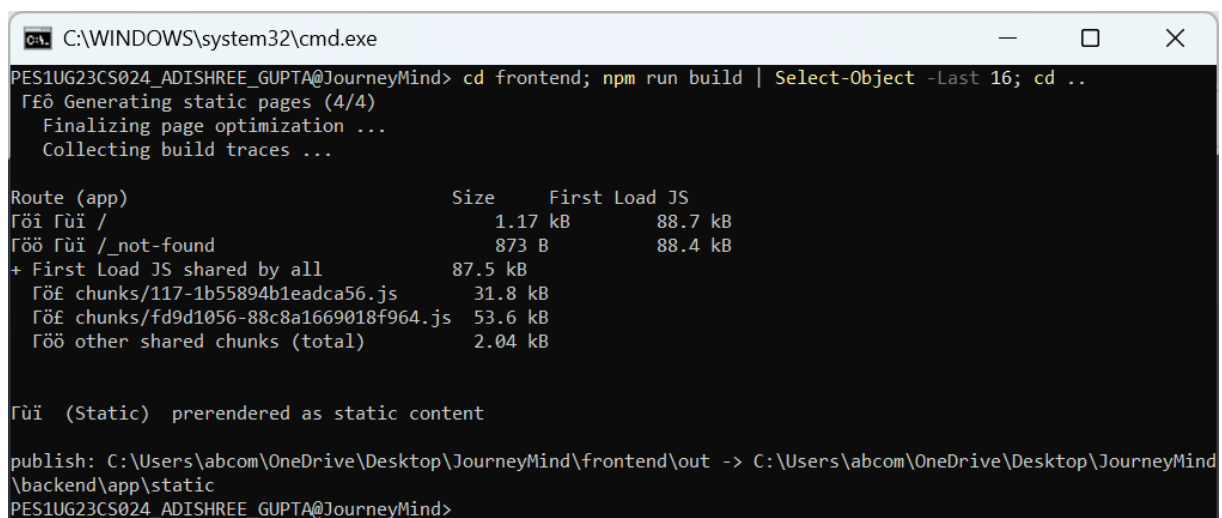


```

JourneyMind DBMS practical
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind> Get-ChildItem -Name frontend\app, frontend\src
layout.jsx
page.jsx
api.js
App.jsx
Book.jsx
Compare.jsx
Enterprise.jsx
Insights.jsx
Journey.jsx
JourneyMap.jsx
modes.js
styles.css
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind> Get-ChildItem -Name backend\app\static
_next
404.html
index.html
index.txt
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind>

```

Figure 33: The Next.js application folder, the reused components and the built output



```

C:\WINDOWS\system32\cmd.exe
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind> cd frontend; npm run build | Select-Object -Last 16; cd ..
  ðŸšš Generating static pages (4/4)
    Finalizing page optimization ...
    Collecting build traces ...

Route (app)                Size      First Load JS
ãŸŸãŸ /                     1.17 kB      88.7 kB
ãŸŸãŸ /_not-found            873 B        88.4 kB
+ First Load JS shared by all      87.5 kB
  ðŸŒš chunks/117-1b55894b1eadca56.js  31.8 kB
  ðŸŒš chunks/fd9d1056-88c8a1669018f964.js  53.6 kB
  ðŸŸãŸ other shared chunks (total)    2.04 kB

ãŸŸãŸ (Static) prerendered as static content

publish: C:\Users\abcom\OneDrive\Desktop\JourneyMind\frontend\out -> C:\Users\abcom\OneDrive\Desktop\JourneyMind\backend\app\static
PES1UG23CS024_ADISHREE_GUPTA@JourneyMind>

```

Figure 34: The Next.js production build completing

Book a Ride

The main screen. The user enters a start and a destination and receives every practical option, cheapest first, with its fare, duration and availability.

JourneyMind: A MultiCommute Decision System

JourneyMind Book the ride that actually gets you there.

[Book a ride](#)[Insights](#)[Intelligence](#)[Journey planner](#)[Enterprise](#)

PES1UG23CS024_Adishree_Gupta

FROM

Wipro Campus, Doddakannelli (Sarjap)

TO

PES University, RR Campus (100 Feet I

Find rides

☒ Demo mode — reproducible booking sequence

Rides to PES University, RR Campus (100 Feet Ring Road)

6 available now

Bus

· BMTC

91 min · 21.2 km

Moderate availability

₹153

Start trip

Metro

· Namma Metro

55 min · 20.6 km

Moderate availability

₹167

Start trip

Bike taxi

· Rapido

61 min · 4 min pickup · 21.3 km

High availability

₹215

Book now

Auto

· Namma Yatri

75 min · 6 min pickup · 21.3 km

High availability

₹320

Book now

Auto

· Metered auto

75 min · 6 min pickup · 21.3 km

High availability

₹371

Book now

Figure 35: The Book a ride screen showing six travel options compared side by side

Booking and Retry

Pressing Book now simulates the booking, including the possibility that a driver accepts and then cancels. The user may try again up to four times, and each retry is priced higher because the market has just demonstrated that it is tight.

JourneyMind: A MultiCommute Decision System

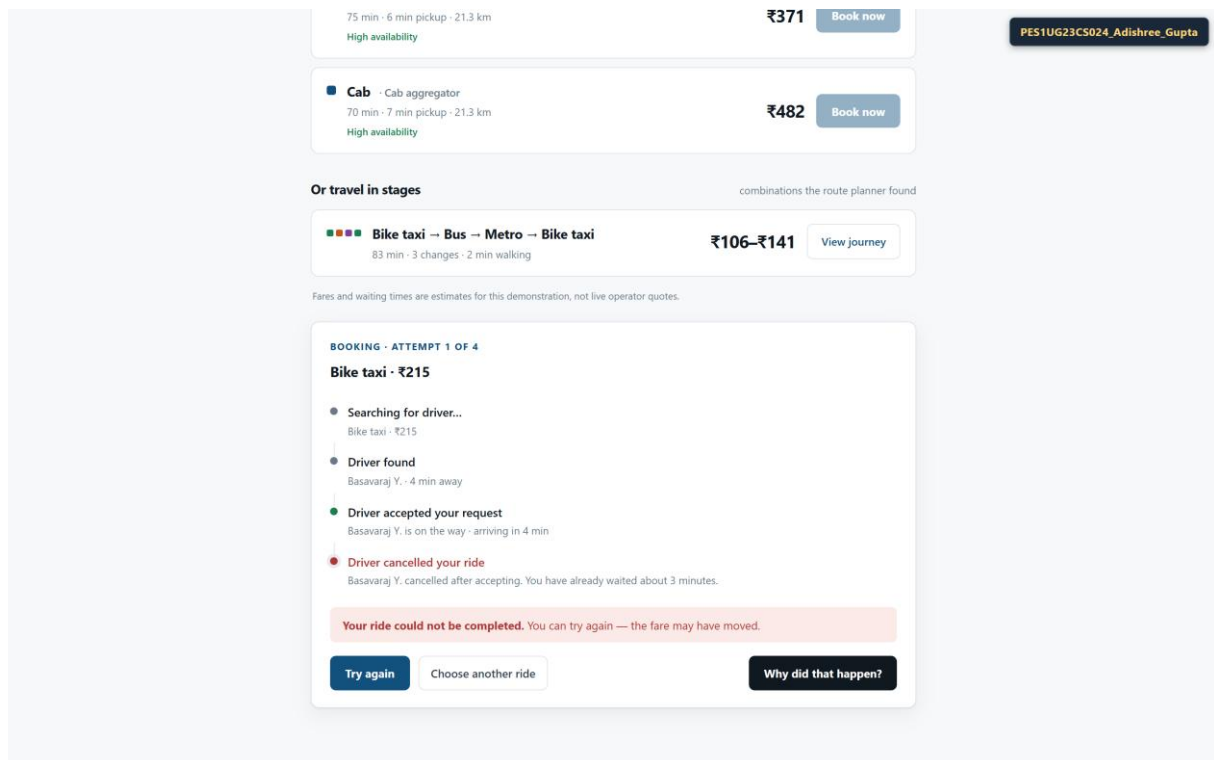


Figure 36: A real booking attempt: driver found, accepted, then cancelled, with the retry options offered

Intelligence: Advertised Price Against Expected Price

This screen prices each option twice: the advertised fare, and the fare the traveller can expect to pay once cancellation and re-booking are taken into account.

JourneyMind: A MultiCommute Decision System

The screenshot displays the JourneyMind web application interface. At the top, the header includes the logo "JourneyMind" with the tagline "Book the ride that actually gets you there.", navigation links for "Book a ride", "Insights", "Intelligence" (highlighted), "Journey planner", and "Enterprise", and a user profile button labeled "PES1UG23CS024_Adishree_Gupta". Below the header, a "DEMO / ESTIMATED DATA" badge is visible. The main search area contains input fields for "FROM" (Wipro Campus, Doddakannelli (Sarjapur Rc)), "TO" (PES University, RR Campus (100 Feet Ring I)), "BUDGET (₹)" (any), and "MAX TIME" (any), followed by a "Compare" button. Below these fields, a "PRIORITY" section shows four buttons: "Cheapest", "Balanced" (selected), "Fastest", and "Most reliable", with a descriptive text "Cost and time together, with a reliability floor". The main content area features a message: "The cheapest fare is not the cheapest trip. Every app shows you an advertised fare. None of them tells you that a third of those bookings fall through, that you lose six minutes finding out, and that the replacement ride costs more. Enter a trip and see what each option is really expected to cost."

Figure 37: Every option priced twice: advertised fare and expected cost

Journey Planner

A complete multimodal journey drawn on a map, with each leg listed in order. The transit lines shown come from the network stored in Neo4j.

JourneyMind: A MultiCommute Decision System

JourneyMind Book the ride that actually gets you there.

Book a ride

Insights

Intelligence

Journey planner

Enterprise

PES1UG23CS024_Adishree_Gupta

DEMO / ESTIMATED DATA

Where are you going?

One question, one answer — the best complete journey inside the money and time you actually have.

FROM

Wipro Campus, Doddakannelli (Sarjapur Road)

Using Wipro Campus, Doddakannelli (Sarjapur Road)

TO

PES University, RR Campus (100 Feet Ring Road)

Using PES University, RR Campus (100 Feet Ring Road)

DEPARTURE

Leaving now

08:43:35 Kolkata

12-09-2026 08:43

Live: priced for this minute on the Bengaluru clock, and refreshed every minute.

BUDGET (₹)

250

MAX TIME (MIN)

120

PREFERENCE

Cheapest

Balanced

Fastest

Set your own priorities

Nobody tells you the best *combination* of rides.

Ride apps price one ride. Map apps ignore your budget. JourneyMind looks at the whole trip — bike taxi, auto, cab, metro, bus — and finds the best complete journey inside the money and the time you actually have.

Run the demo: Wipro Sarjapur Rd → PES University

Study area: Bengaluru — Purple/Green/Yellow Line corridor

Figure 38: The Journey planner with a multimodal route drawn on the map

23CS024 - 23CS624 - 23CS366

Page 45 / 49

JourneyMind: A MultiCommute Decision System

Insights

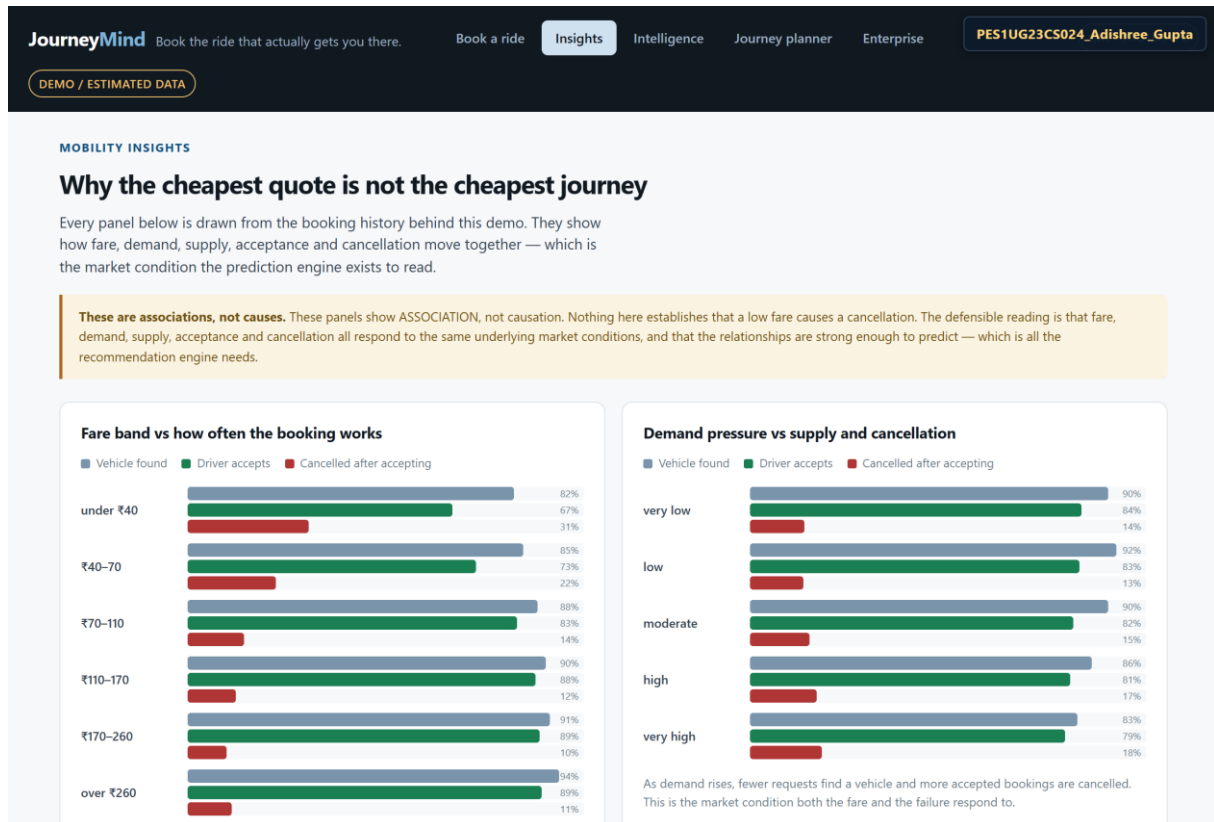


Figure 39: The Insights screen, explaining when and why bookings fail

Enterprise Dashboard

The organisation view. Total spend, booking success rate, cancellation rate and the cost of failed bookings, with breakdowns by campus, team and provider. This screen is the MySQL booking history being filtered and aggregated, cached through Redis, and displayed. It requires an API key.

JourneyMind: A MultiCommute Decision System

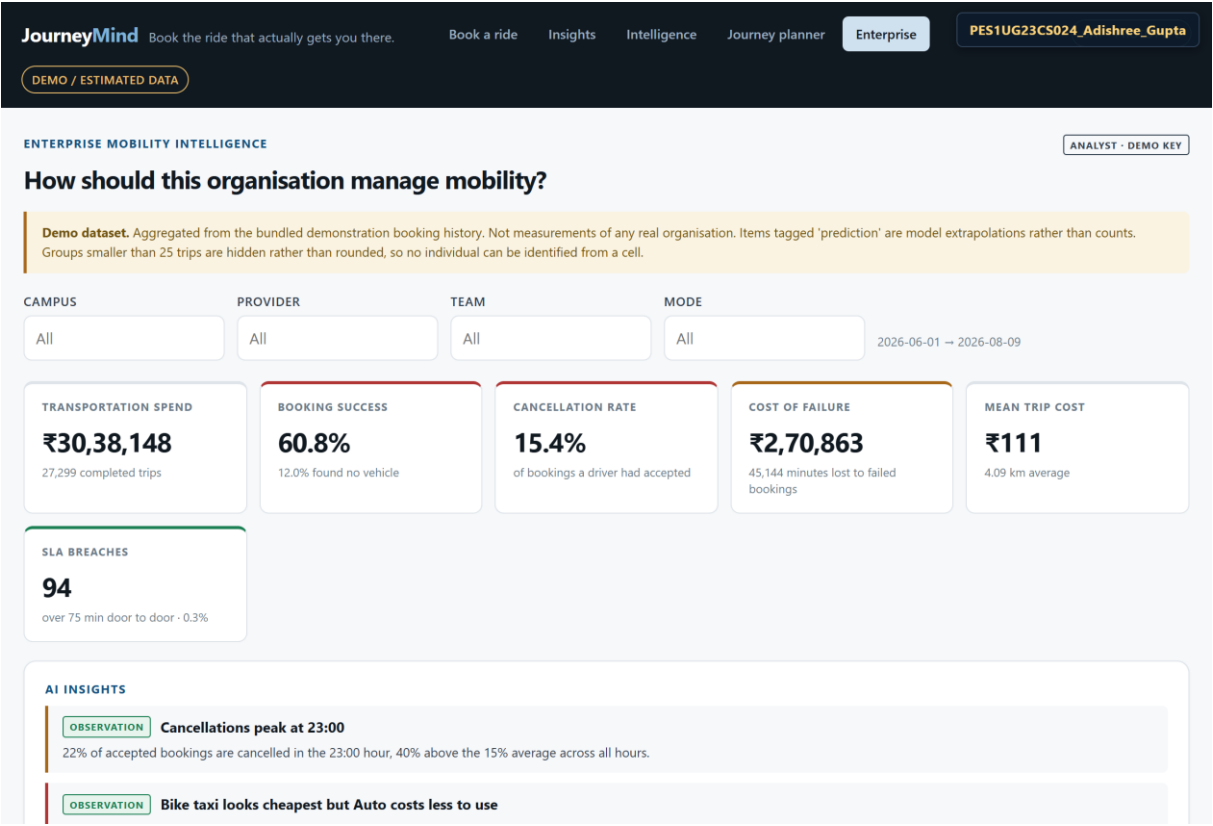


Figure 40: The Enterprise dashboard, aggregated from the MySQL booking history

Conclusion

JourneyMind answers a practical question — what is a sensible way to make this journey — and answers it honestly, including what happens when a booking fails. The project goes beyond storing and retrieving records: it judges whether each travel mode is practical for the distance before comparing options, prices the cost of failure, plans routes across a multimodal network, and records every decision so that it can be audited.

From a database point of view, the main conclusion is that the right database depends on the question being asked. MySQL holds the structured, permanent, related records and answers questions about how much and how often, using joins, aggregation and indexes. Redis holds the fast and short-lived data and answers questions by key, letting the data that should expire do so. Neo4j holds the transport network and answers questions about how places connect. Each choice follows from the shape of the data rather than from a wish to use three technologies, and the application still runs when any one of them is removed, which is the clearest evidence that none is decoration.

Verified Outcomes

- **13 of 13 database structures** hold at least 100 meaningful records after a clean initialisation, verified by querying the live databases.
- **315 automated tests pass** and 1 is skipped for a documented, data-dependent reason.
- **26 of those tests** exercise MySQL, Redis and Neo4j directly, rather than only checking that a connection opens.
- Referential integrity holds with zero orphaned rows, and the graph contains no disconnected nodes.

Future Enhancements

- Replace the simulated availability model with live operator data.
- Full user accounts with individual travel history, in place of API keys.
- Extend the study area beyond the current corridor; the graph model already scales, only the bundled data is limited.
- Add database triggers to record an audit entry automatically on write, rather than from application code.
- Learn each traveller's own cost and time trade-off from their past choices instead of using fixed presets.
- Use the graph to re-route around a disruption when a line is closed.

List of Software Tools Used

Tool	Role in the project	Purpose
Next.js 14 (React 18)	Front end	The screens the user interacts with, built with the App Router and exported as static files.
FastAPI (Python 3.12)	Back end / API	Receives requests from the front end, runs the application logic, and talks to the databases.
MySQL 8.0.46	SQL database	Permanent structured records: zones, booking history, booking sessions, attempts and the audit trail.
Redis 8.10.1	NoSQL key-value database	Fast temporary data: cached place lookups, live booking sessions, cached dashboard answers and the recent-decision list.
Neo4j 5.26 Community	NoSQL graph database	The transport network as stops and the connections between them.
Leaflet + OpenStreetMap	Mapping	Drawing the journey and the transit lines on a map.
NumPy	Computation	Journey search and the serving-time model arithmetic.
pytest	Testing	Automated unit, API and database integration tests.
Playwright	UI testing	Drives a real browser to verify the application end to end and capture screenshots.
Git and GitHub	Version control	Source code history and hosting.

Table 14: Technology stack

References, URLs

- [1] Next.js Documentation. Vercel. <https://nextjs.org/docs>
- [2] FastAPI Documentation. Sebastian Ramirez. <https://fastapi.tiangolo.com>
- [3] MySQL 8.0 Reference Manual. Oracle Corporation. <https://dev.mysql.com/doc/refman/8.0/en/>
- [4] Redis Documentation - Data Types and Commands. Redis Ltd. <https://redis.io/docs/latest/develop/data-types/>
- [5] Neo4j Cypher Manual. Neo4j Inc. <https://neo4j.com/docs/cypher-manual/current/>
- [6] OpenStreetMap contributors. Map data licensed under the Open Database Licence (ODbL). <https://www.openstreetmap.org/copyright>
- [7] Elmasri, R. and Navathe, S. B. Fundamentals of Database Systems. Referenced through the Unit 4 course material on NoSQL systems, key-value stores and graph databases.
- [8] Course material: UE24CS351A Database Management Systems, Unit 4 - Introduction to NoSQL Systems (Lecture 46), Redis (Lecture 49), Graph Databases and Neo4j (Lectures 50 and 51). PES University.